SPACE FACULTY

Let’s learn about **satellites**!

Space mission and **satellite** design course

Hands-on Slides

2NDSpace

Satellite Design & Integration

Hands-on 1

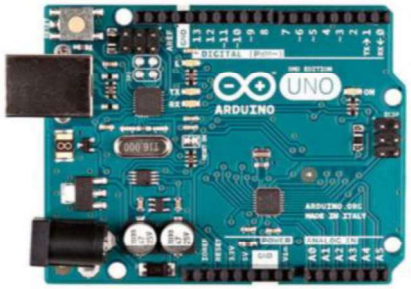
2NDSpace

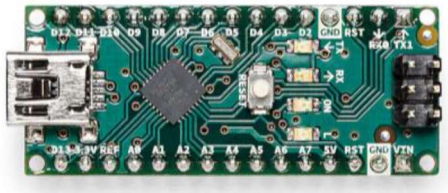
Why Arduino?

Arduino is an open-source platform.

It includes:

- Hardware (UNO, Nano, Mega...)





- Software (Arduino IDE)

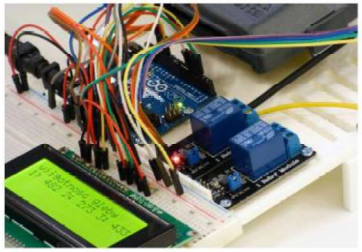
It is a simple way to start learning electronics and programming.

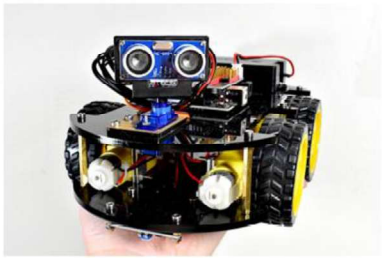
Why We Use Arduino

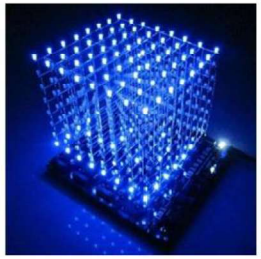
Arduino is:
- Accessible
- Low cost
- Easy to use
- Ideal for rapid prototyping

It allows you to:
- Test ideas quickly
- Build real hardware projects
- Learn by doing

Large online community → many libraries and examples available.







What Is Inside an Arduino?

An Arduino board includes:

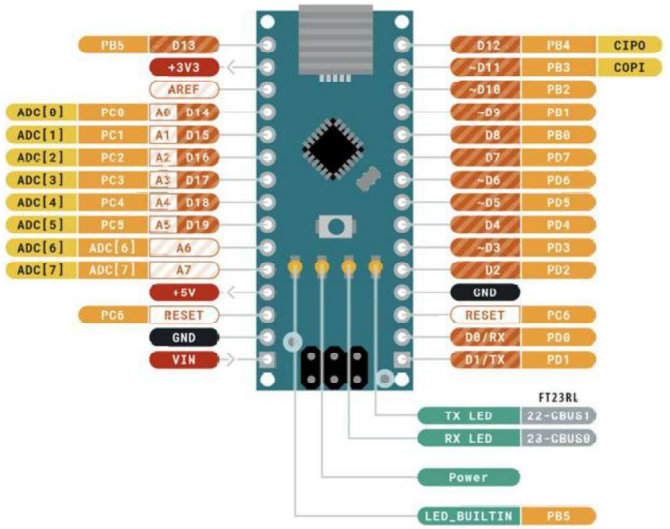
Microcontroller
- The “brain” of the board
- Contains CPU, memory, and peripherals

Input / Output pins
- Used to connect sensors and actuators

Power supply
- USB
- External supply
- Battery

It is a small embedded system.

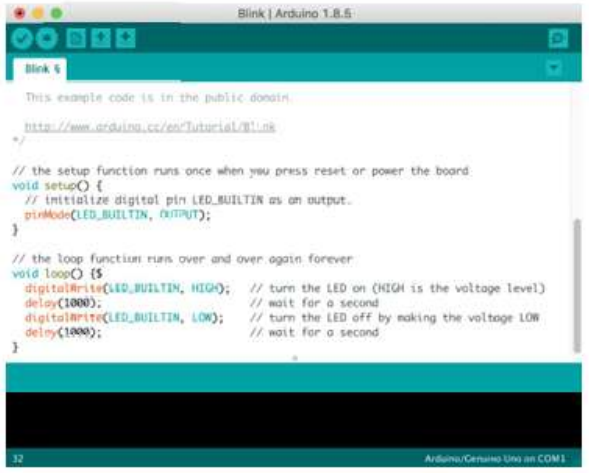
Arduino Nano



Programming Arduino

Arduino is programmed via:
- USB cable
- Arduino IDE

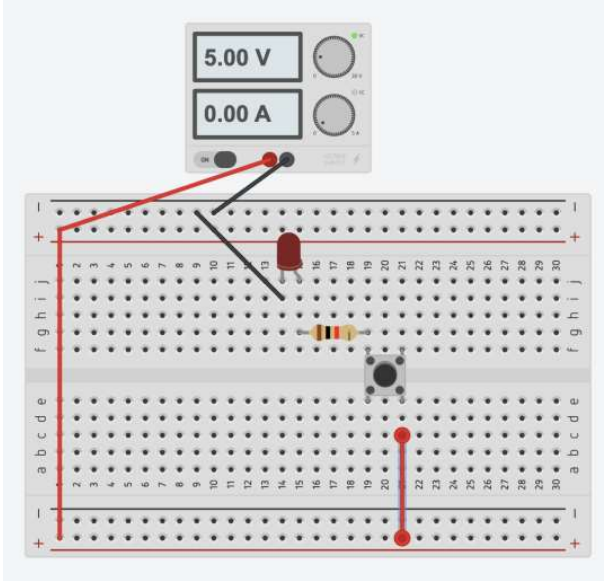
You write code on your computer and upload it in seconds.

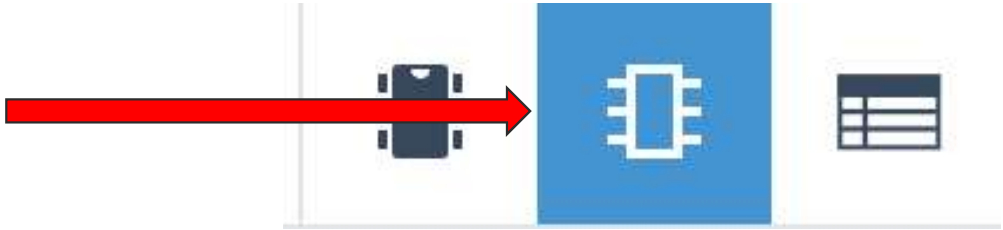


Tinkercad

Recreate this circuit in Tinkercad:

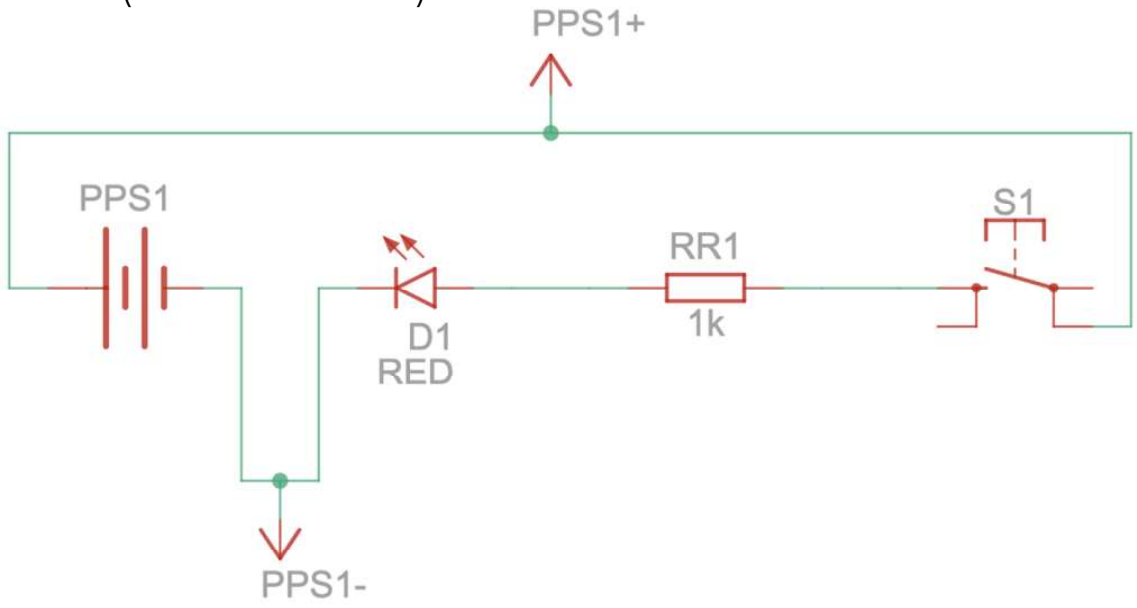
- Let's go on www.tinkercad.com
- Login with your Google account
- Click on "Create" then "Circuit"
- Pick the components and make the circuit
- When done, click on the top right corner icon that shows you the schematic view



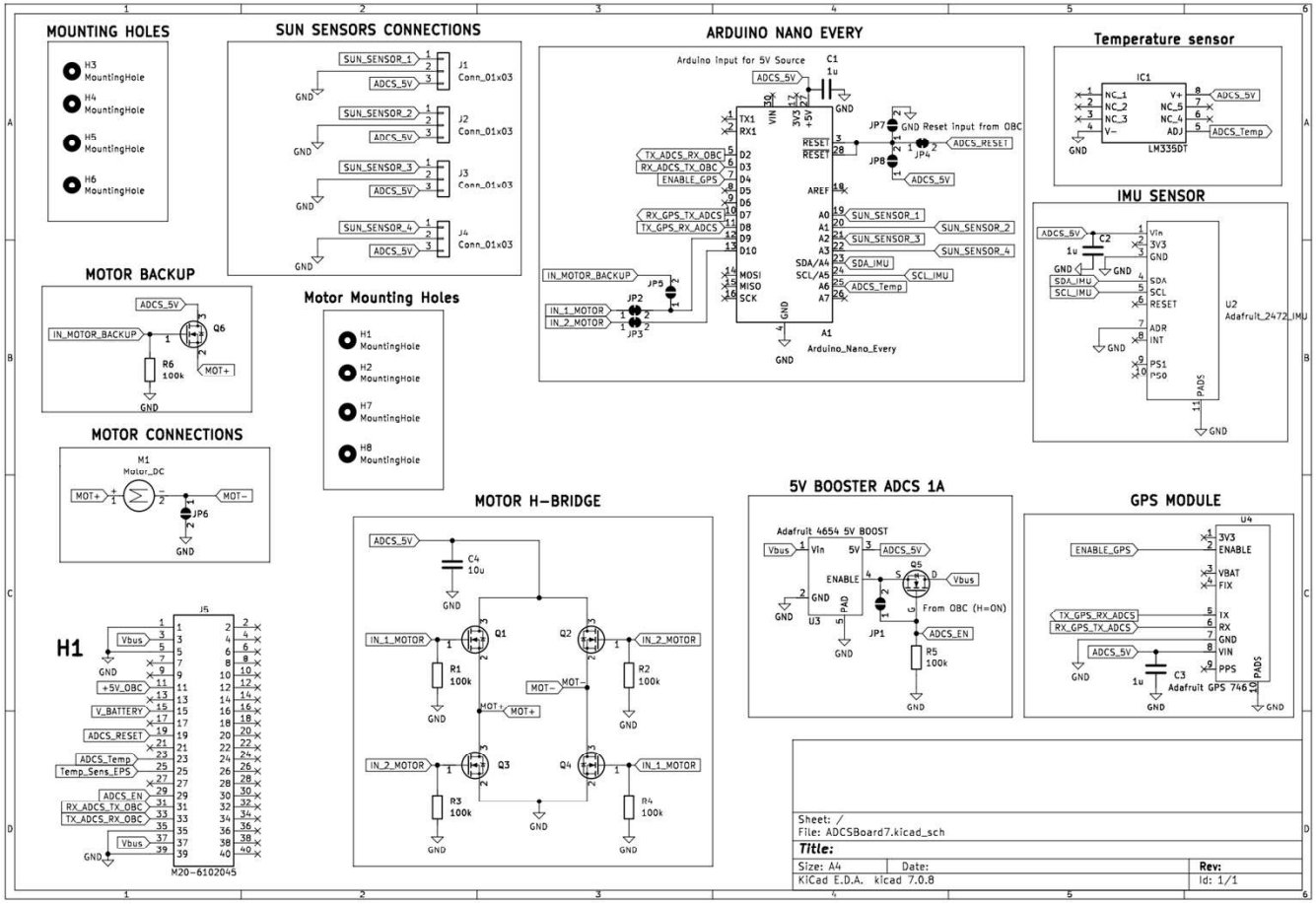


Schematics of an LED, Resistor and Button Circuit

This is how the circuit we created can be drawn from an electrical point of view (schematics).

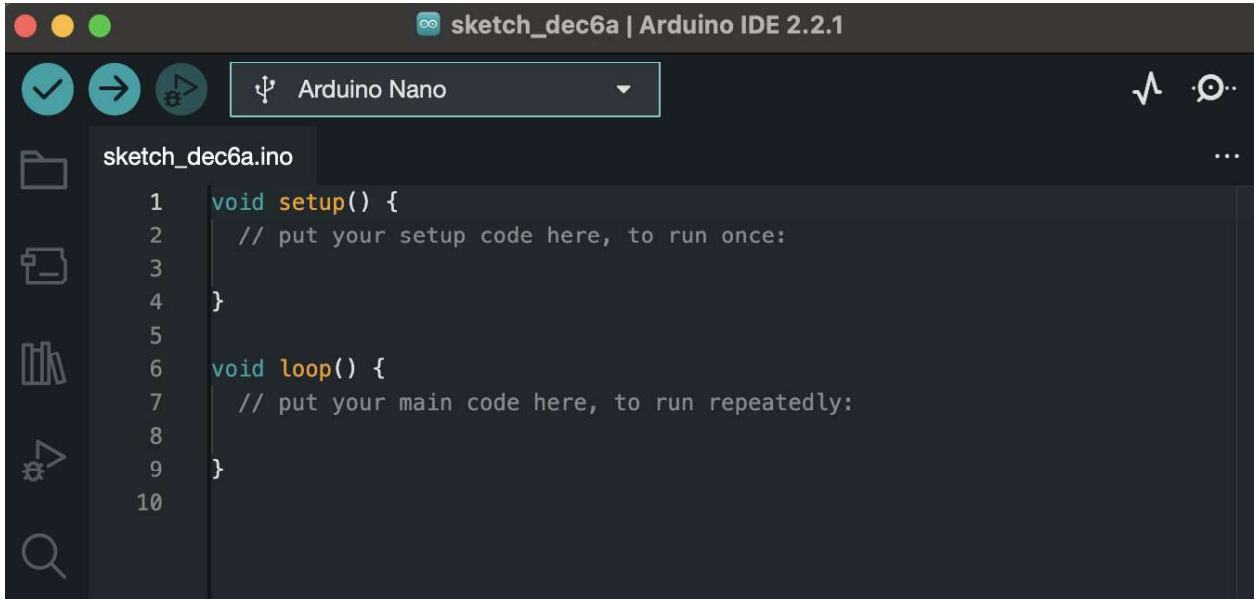


Satellite Boards Schematics



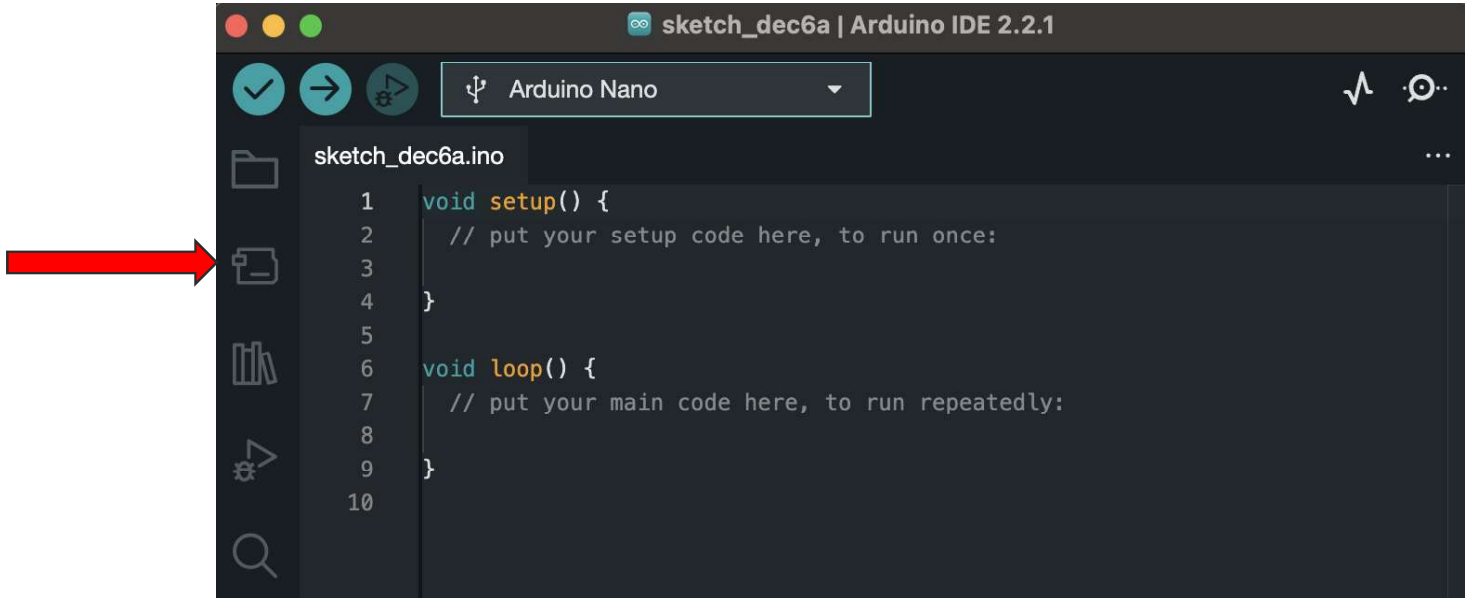
Arduino Programming Set Up

- Let's go on Arduino website https://github.com/arduino/arduino-ide/releases/tag/2.3.4
- Find the 2.3.4 version of Arduino IDE for either Mac or Windows.



Arduino Programming Set Up

- Go on “Boards Manager”.
- Search “megaAVR” and install it.

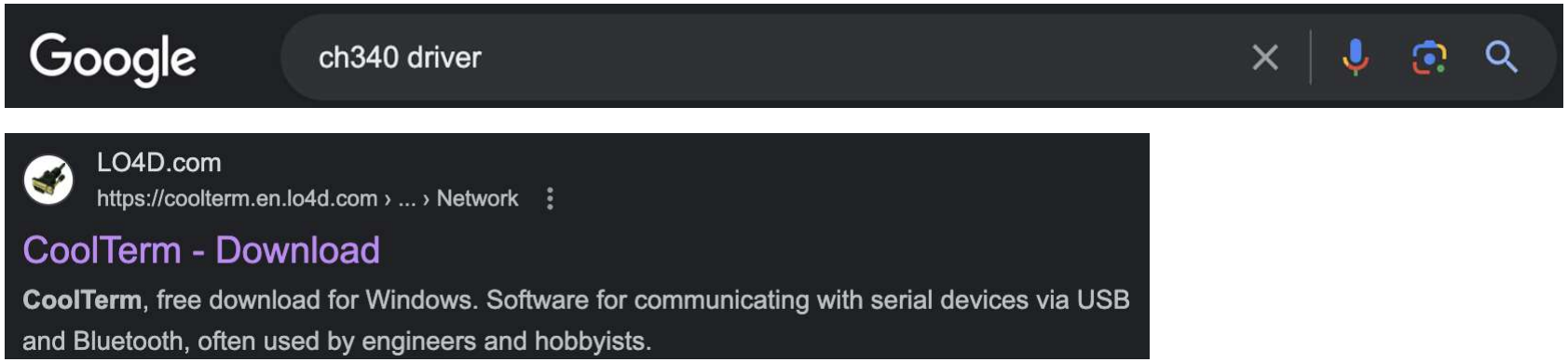


Arduino Programming Set Up

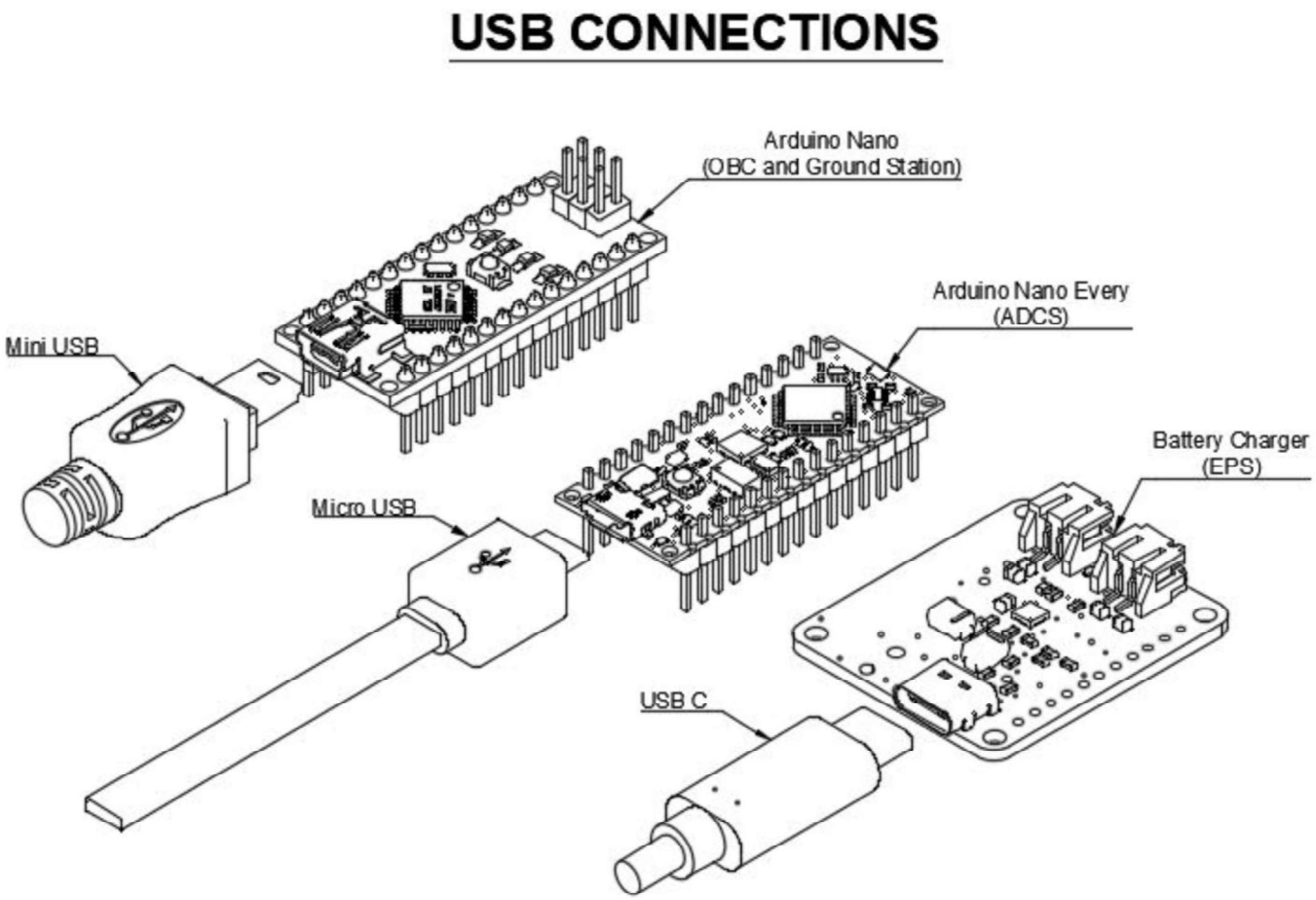
- Go on “Library Manager”.
- Install the first library you see appearing.

Arduino Programming Set Up

- Now locate your “libraries” folder inside the “Arduino” folder in that got installed with the IDE.
- Copy and paste there the library folders we are going to use in programming the satellite.
- For Windows users, install a driver called “CH340” online to recognize Arduino Nano.
- Search and install a program called “CoolTerm”.



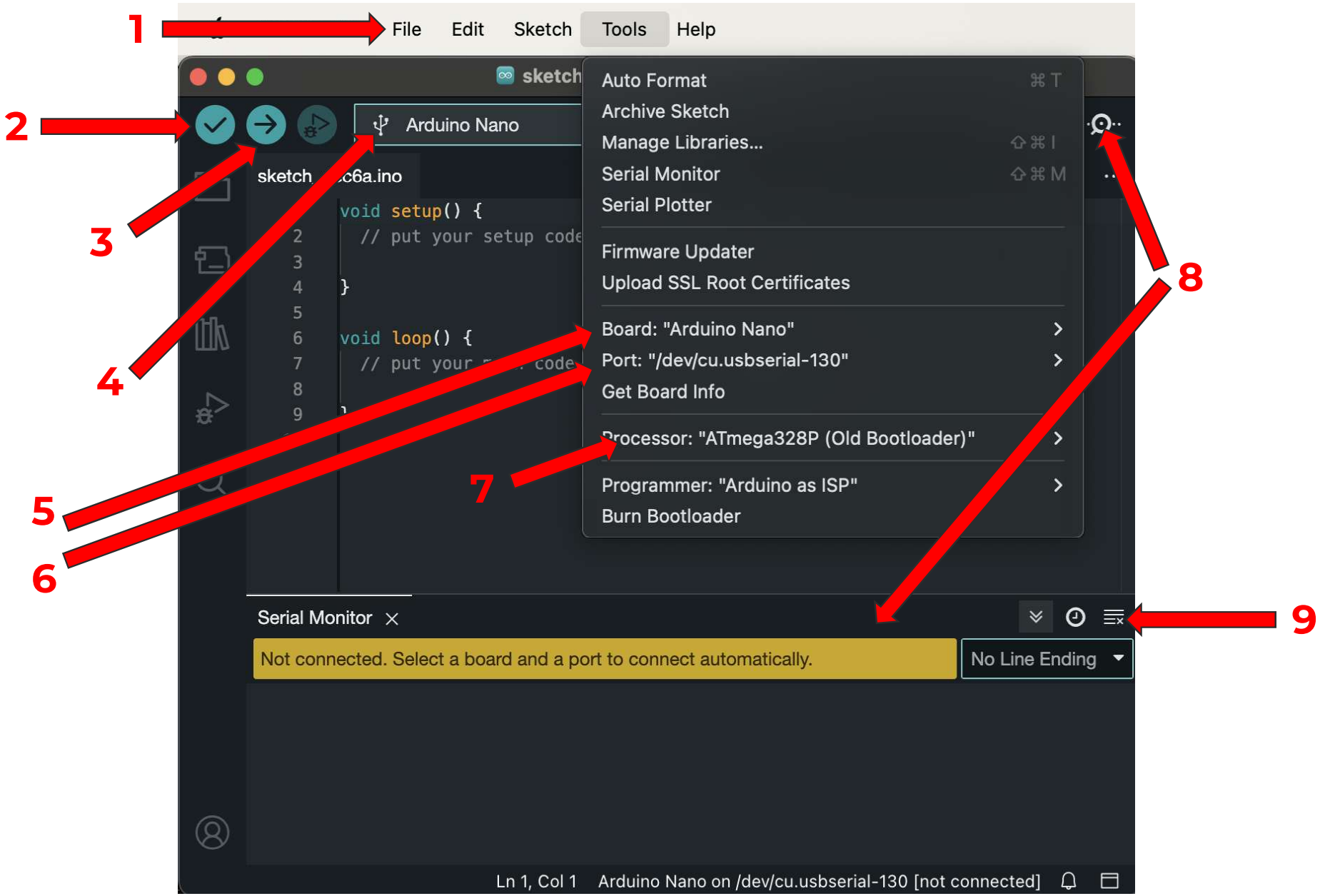
Arduino Nano vs Arduino Nano Every



Introduction to Arduino IDE

Basic interface:

1. Create a new sketch, save, open examples.
2. "Verify" checks the syntax and compiles the code to identify errors without uploading it to the board.
3. “Upload” transfers the compiled code from your computer to the Arduino board, making it ready to execute the program on the hardware.
4. Board and COM Port selection
5. Board selection (either Arduino Nano or Nano Every)
6. COM Port selection
7. For Arduino Nano (GS and OBC), select “Old Bootloader”.
8. Open/Close Serial Monitor
9. Clear Serial Monitor


1
File
Edit
Sketch
Tools
Help
2
3
4
5
6
7
8
9
sketch
Arduino Nano
Auto Format ⌘T
Archive Sketch
Manage Libraries... ⇧⌘I
Serial Monitor ⇧⌘M
Serial Plotter
Firmware Updater
Upload SSL Root Certificates
Board: "Arduino Nano"
Port: "/dev/cu.usbserial-130"
Get Board Info
Processor: "ATmega328P (Old Bootloader)"
Programmer: "Arduino as ISP"
Burn Bootloader
void setup() {
// put your setup code
}
void loop() {
Serial Monitor
Not connected. Select a board and a port to connect automatically.
No Line Ending
Ln 1, Col 1
Arduino Nano on /dev/cu.usbserial-130 [not connected]

Satellite Design & Integration

Hands-on 2

2NDSpace

Satellite Kit Overview

Open your satellite boxes!

Let’s place all the components and little bags on the tables to check everything we are going to need is there:

- Satellite subsystems (electronic boards).
- Hardware parts (screws, bolts, wires…).

Screwdriver Set

The screwdriver set includes a variety of bits to accommodate the diverse types of screws used in different devices and applications.

Various screws exist due to the need for specialized fastening solutions in manufacturing and design. Using the right bit for the corresponding screw head is crucial to prevent damage to screws or the bits themselves.

Screwdriver Set

We have for instance:

- Hex screws, common in furniture and electronics.
- Phillips-head screws offering efficient torque transmission in many applications.
- Torx screws that provide enhanced security and durability.

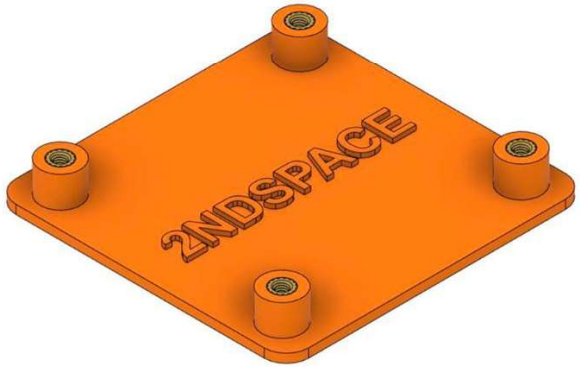
EPS Testing!

By looking at the EPS schematics, use the continuity function of the multimeter to check ground connections around the board.

- Find and show me at least 5 ground (GND) connections using the multimeter.
- Be careful not to short circuit anything with the tips of multimeter probes!

Building the Ground Station (GS)

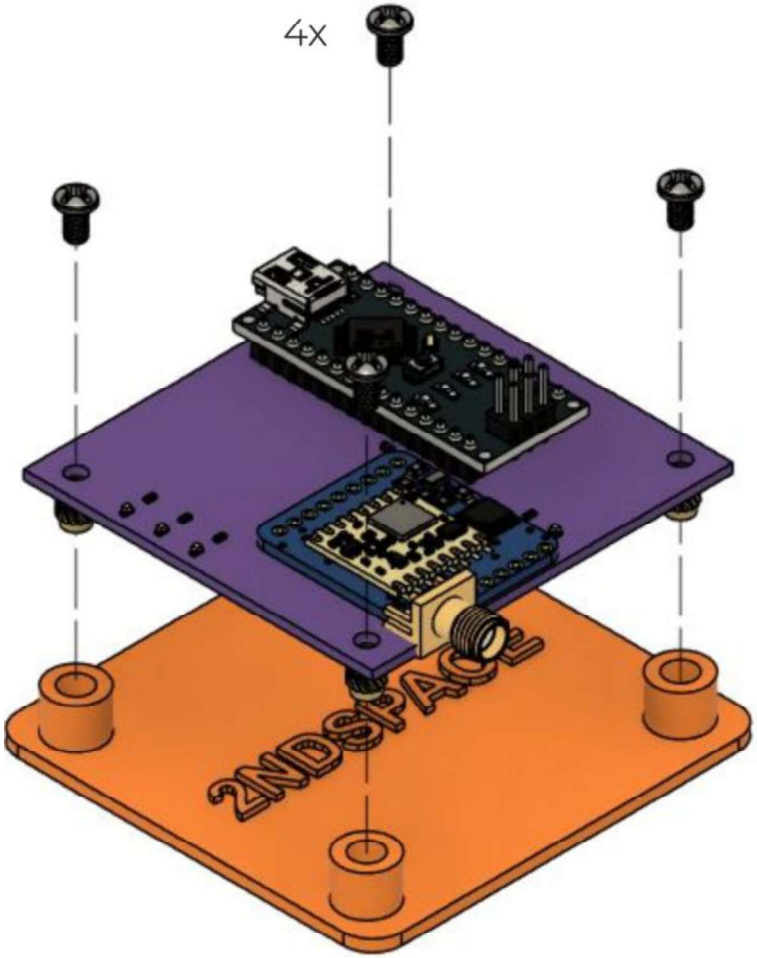
4xM3x6



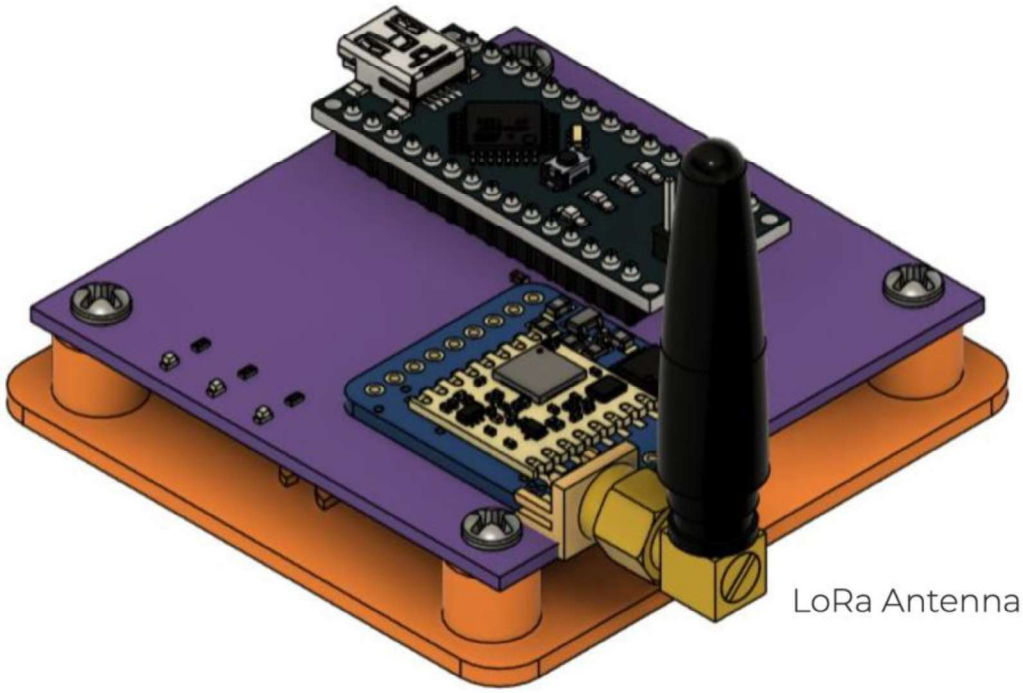
1x Ground Station Support

1x Ground Station Board

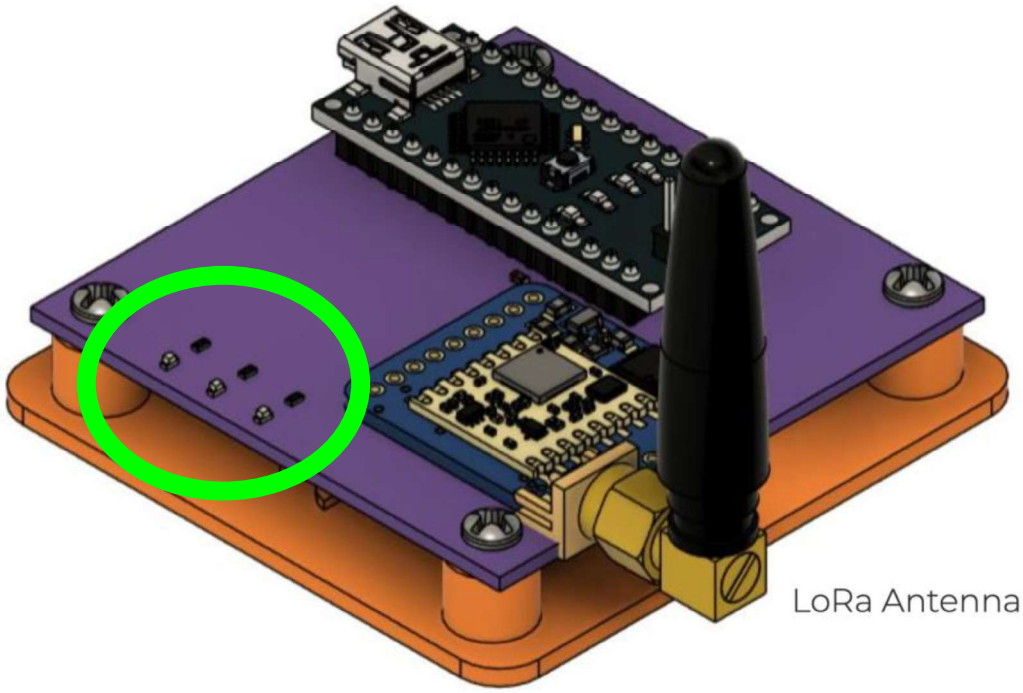
Building the Ground Station (GS)



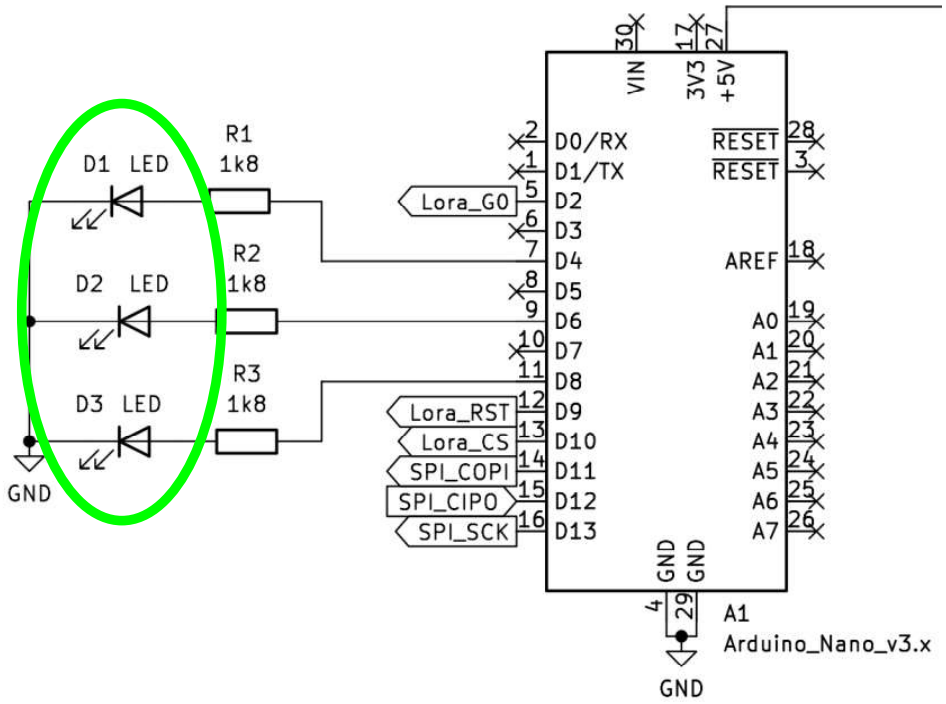
Building the Ground Station (GS)



Building the Ground Station (GS)



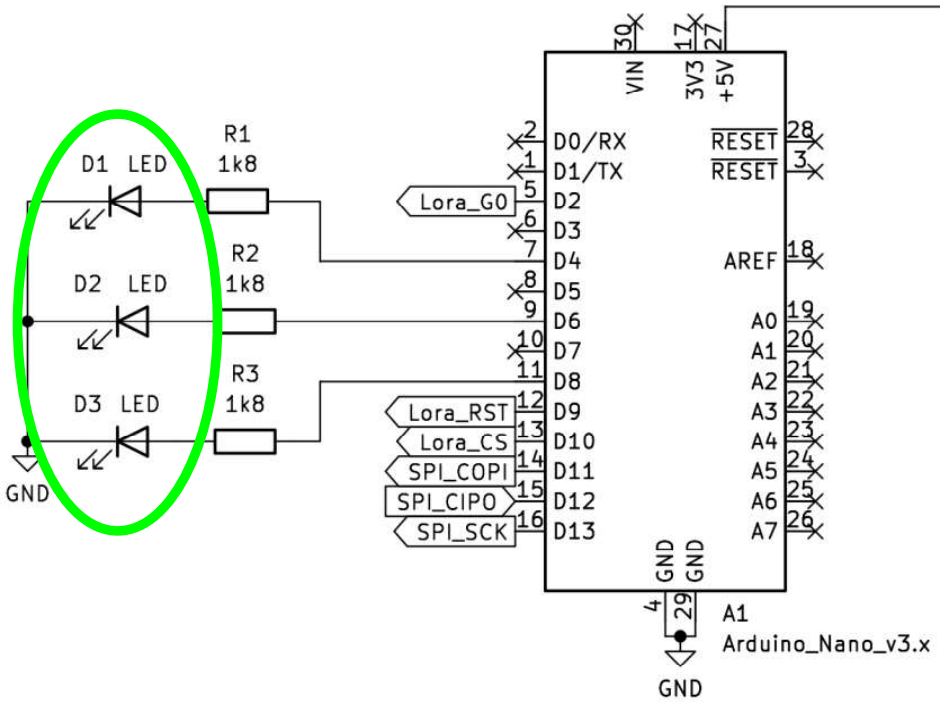
Building the Ground Station (GS)



Building the Ground Station (GS)

In the GS we have SMD LEDs.

SMD components are smaller, mounted directly on the PCB surface, and suitable for automated production.
PTH components are larger, have leads through holes, and are often manually soldered.



Your First Code

What do you think this code does?

```
1  #define LED 4
2
3  void setup() {
4    pinMode(LED, OUTPUT);
5  }
6
7  void loop() {
8    digitalWrite(LED, HIGH);
9  }
```

Your First Code

Open a new sketch in Arduino IDE and copy this code there, let's upload it together!

```
#define LED 4

void setup() {
  pinMode(LED, OUTPUT);
}

void loop() {
  digitalWrite(LED, HIGH);
}
```

IMPORTANT INFORMATION! 

If you mistakenly tell an INPUT pin to act like an OUTPUT in your Arduino code and it tries to send or receive current, you could end up damaging your Arduino.

Coding Mistake: If you accidentally make an INPUT pin behave like an OUTPUT in your code, it means you're giving it the wrong instructions. INPUT pins are meant to sense signals, while OUTPUT pins are used to send signals or power devices like LEDs. If you mix them up in your code and tell an INPUT pin to send current, it can lead to too much current flowing through the pin, possibly harming your Arduino.

Always double check what code are you uploading on the Arduino!

Let’s Start The Coding!

During this workshop, we will cover all the relevant aspects of Arduino programming, using the satellite as an example of a complete platform with inputs, outputs, both digital and analog, sensors, communication modules, and more.

Utilize all available online resources to help you figure out how to complete the exercises, similar to how engineers approach unfamiliar components by consulting documentation.

Please refrain from using ChatGPT or other AI systems, as they may provide solutions that can be misinterpreted, potentially leading to errors or a lack of understanding of how microcontroller programming works.

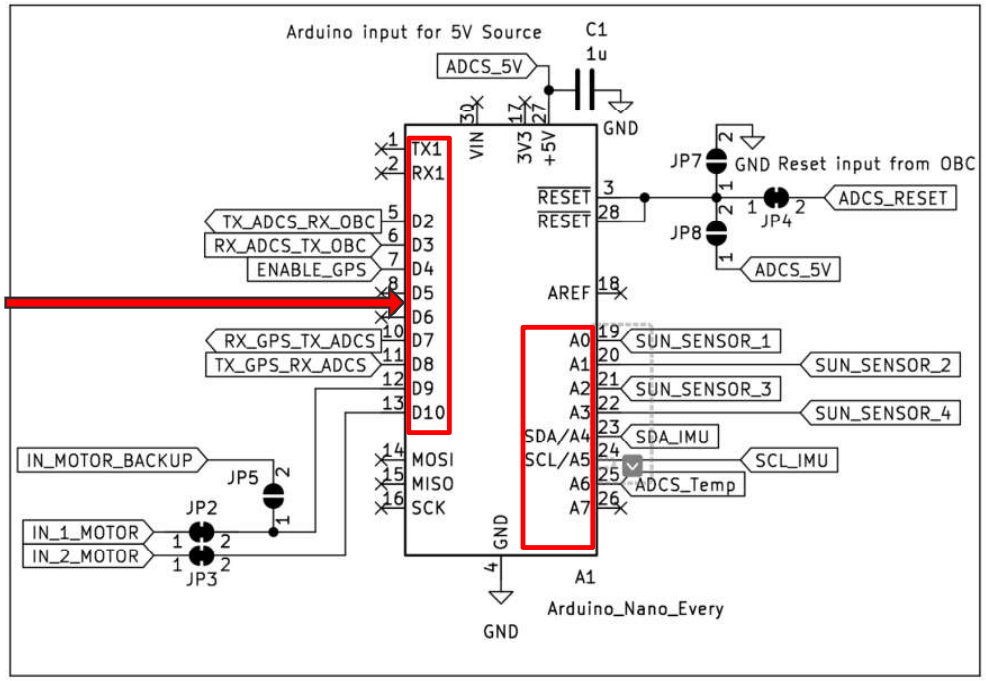
How to Read Arduino Pinouts

Let's consider the ADCS's Arduino:

- When writing the code, you need to read the pinout from here (internal side of the box)
- EXAMPLE

```
#define SUN_SENSOR_1 A0

#define ENABLE_GPS 4
```



Ground Station Exercises - LEDs

By looking at the GS schematics:

- Identify where LED D1 is connected to the Arduino.
- Write a code to turn it on.

Ground Station Exercises - LEDs

By looking at the GS schematics:

- Identify where LED D1 is connected to the Arduino.
- Write a code to turn it on.
- Turn on LED D2 and D3 as well.

Ground Station Exercises - LEDs

By looking at the GS schematics:

- Identify where LED D1 is connected to the Arduino.
- Write a code to turn it on.
- Turn on LED D2 and D3 as well.
- While keeping D2 and D3 on, write a code to blink the LED D1 (on and off) for 500ms. Use the **delay()** function for that.

Ground Station Exercises - LEDs

By looking at the GS schematics:

- Identify where LED D1 is connected to the Arduino.
- Write a code to turn it on.
- Turn on LED D2 and D3 as well.
- While keeping D2 and D3 on, write a code to blink the LED D1 (on and off) for 500ms. Use the **delay()** function for that.
- Write a function to make LEDs blink all together. Hint:

```
5  void loop() {
6    blink(500); //call the function here
7  }
8
9  void blink(int duration) {
10   //write the code here
11 }
```

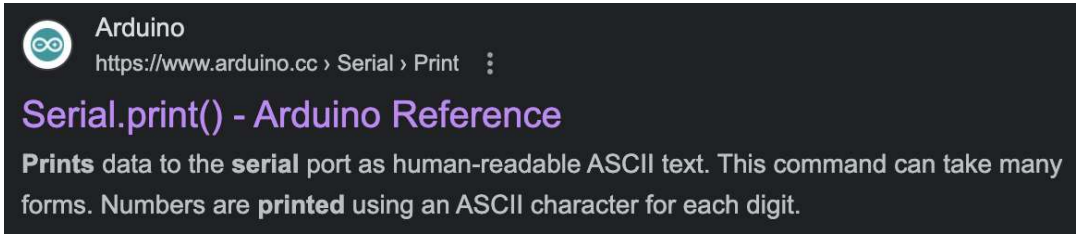
Ground Station Exercises - LEDs

By looking at the GS schematics:

- Identify where LED D1 is connected to the Arduino.
- Write a code to turn it on.
- Turn on LED D2 and D3 as well.
- While keeping D2 and D3 on, write a code to blink the LED D1 (on and off) for 500ms. Use the **delay()** function for that.
- Write a function to make LEDs blink all together.
- Blink D1, D2 and D3 sequentially with a second delay between each other and print in serial monitor which LED is ON in that moment. Use **Serial.print()** for that.

Ground Station Exercises - LEDs

- ...
- Blink D1, D2 and D3 sequentially with a second delay between each other and print in serial monitor which LED is ON in that moment. Use **Serial.print()** for that, you can find all the instructions on arduino website as per how to use this function.



Ground Station Exercises - LEDs

- ...
- Blink D1, D2 and D3 sequentially with a second delay between each other and print in serial monitor which LED is ON in that moment. Use **Serial.print()** for that, you can find all the instructions on arduino website as per how to use this function.
- Make an LED blink 5 times (1 second delay) using a **for loop** and then keep it off for 5 seconds.

Ground Station Exercises - LEDs

- ...
- Blink D1, D2 and D3 sequentially with a second delay between each other and print in serial monitor which LED is ON in that moment. Use **Serial.print()** for that, you can find all the instructions on arduino website as per how to use this function.
- Make an LED blink 5 times (1 second delay) using a **for loop** and then keep it off for 5 seconds.
- Now make it blink 5 times as before, but it remains off afterwards.

Ground Station Exercises - LEDs

- ...
- Blink D1, D2 and D3 sequentially with a second delay between each other and print in serial monitor which LED is ON in that moment. Use **Serial.print()** for that, you can find all the instructions on arduino website as per how to use this function.
- Make an LED blink 5 times (1 second delay) using a **for loop** and then keep it off for 5 seconds.
- Now make it blink 5 times as before, but it remains off afterwards. Hint: use a **while loop** before the **for loop**.

Ground Station Exercises - LEDs

- ...
- Blink D1, D2 and D3 sequentially with a second delay between each other and print in serial monitor which LED is ON in that moment. Use **Serial.print()** for that, you can find all the instructions on arduino website as per how to use this function.
- Make an LED blink 5 times (1 second delay) using a **for loop** and then keep it off for 5 seconds.
- Now make it blink 5 times as before, but it remains off afterwards. Hint: use a **while loop** before the **for loop**.
- Write a code to turn on one LED if the serial monitor is available (send whatever letter to see if it works). If nothing is sent, the LED should stay off.

Ground Station Exercises - LEDs

- ...
- Write a code to turn on one LED with a CHARACTER (**char**) via serial.

Ground Station Exercises - LEDs

- ...
- Write a code to turn on one LED with a CHARACTER (**char**) via serial. E.g. if I send “A” the LED turns on.
- Write a code to turn on one LED with a STRING (**String**) via serial. E.g. if I send “LED1” the LED turns on.

Ground Station Exercises - LEDs

- ...
- Write a code to turn on one LED with a CHARACTER (**char**) via serial. E.g. if I send “A” the LED turns on.
- Write a code to turn on one LED with a STRING (**String**) via serial. E.g. if I send “LED1” the LED turns on.
- Find a way to turn on each LED (D1, D2, D3) by sending “D1”, “D2” or “D3”. If not called, the LEDs should be off.

Satellite Design & Integration

Hands-on 3

2NDSpace

Satellite Set Up

Let’s start building the 1U Cubesat!

Follow the guide:

- Read carefully and understand each step before proceeding.
- Strictly adhere to the guide's sequence; do not skip or advance independently.

Do not power subsystems until instructed:

- Avoid connecting or powering subsystems prematurely to prevent conflicts. Keep the RBF plugged in.

Use the screwdriver set correctly:

- Use the right size of the screwdriver inserts for each specific screw.
- Avoid over-tightening screws; tighten securely without excessive force to prevent damage.

Satellite Set-up - EPS

First of all: find the RBF (Remove Before Flight) jack and plug it in the EPS

=

Satellite Set-up - EPS

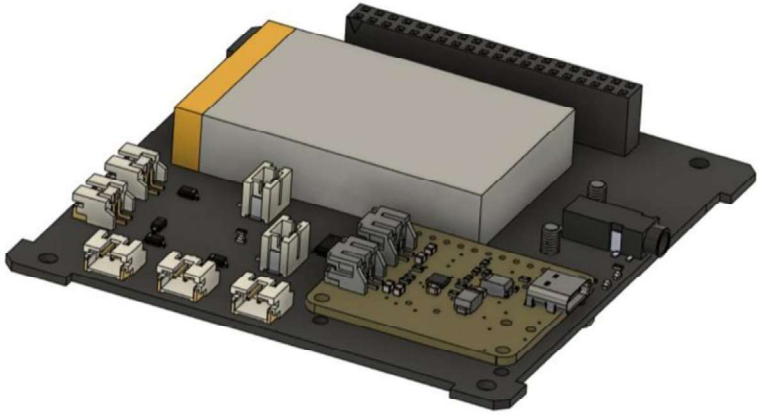
Satellite Set-up - EPS

Let’s attach the battery to the EPS using one or two pieces of double-sided tape and then plug the battery into its connector.

Satellite Set-up - EPS

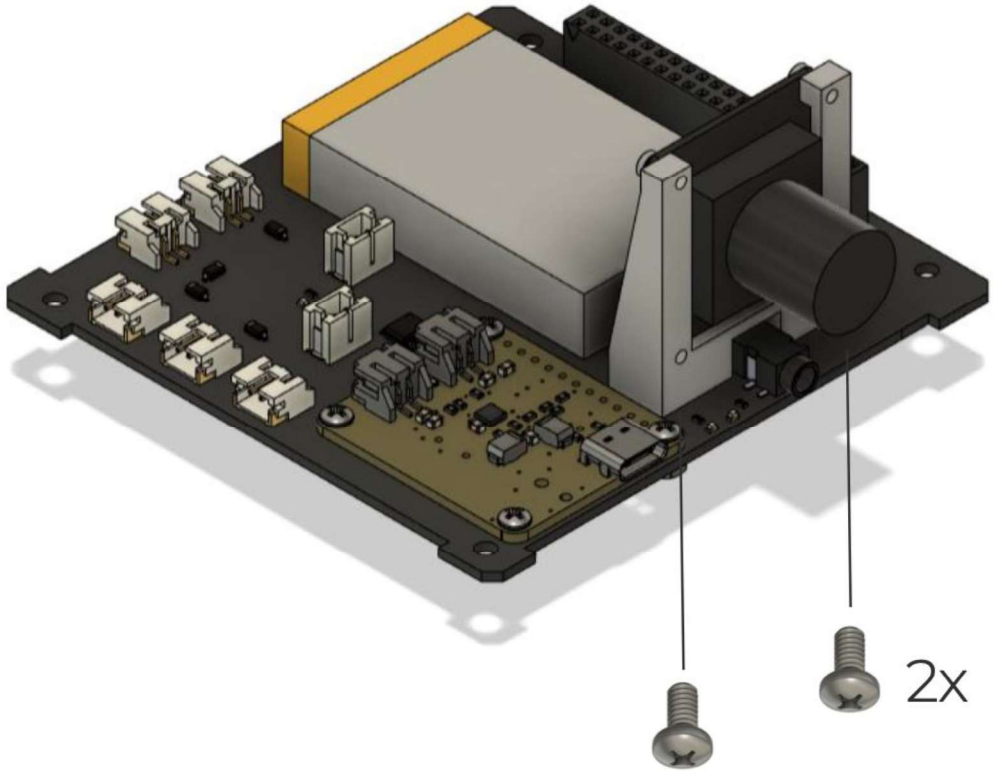
2xM2 x 4

1x Camera



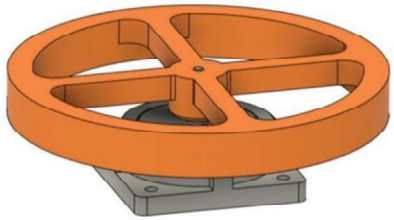
1x EPS Board

Satellite Set-up - EPS



Satellite Set-up - ADCS

4xM2x4

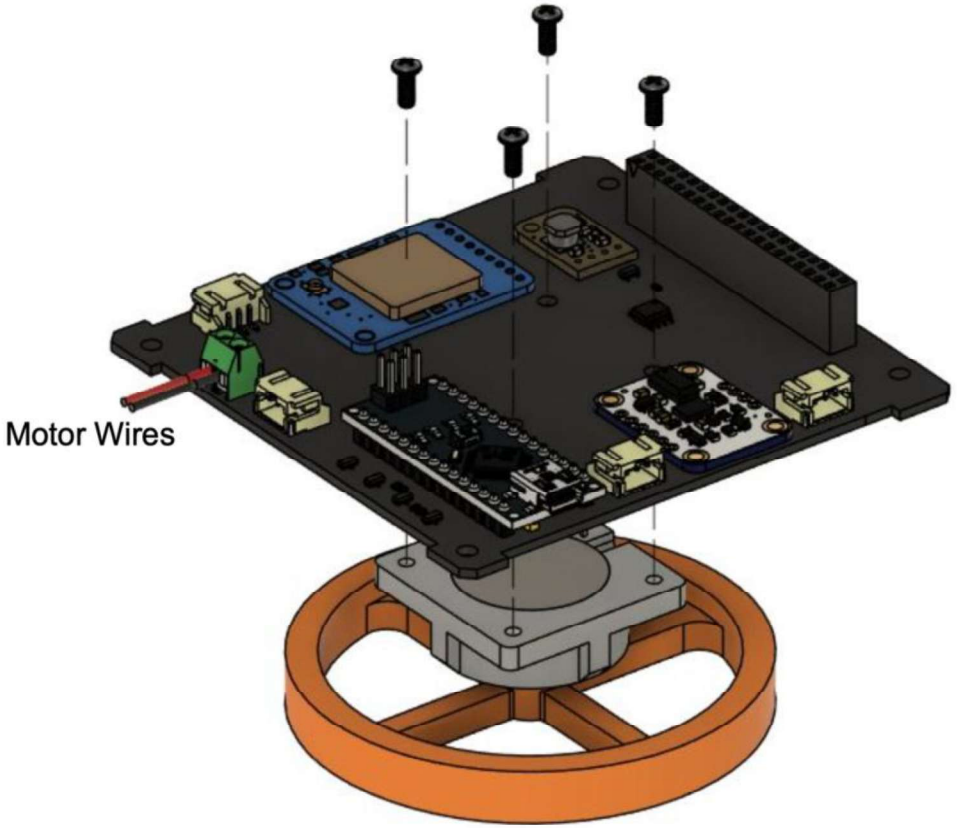


1x Reaction Wheel

1x ADCS Board

Satellite Set-up

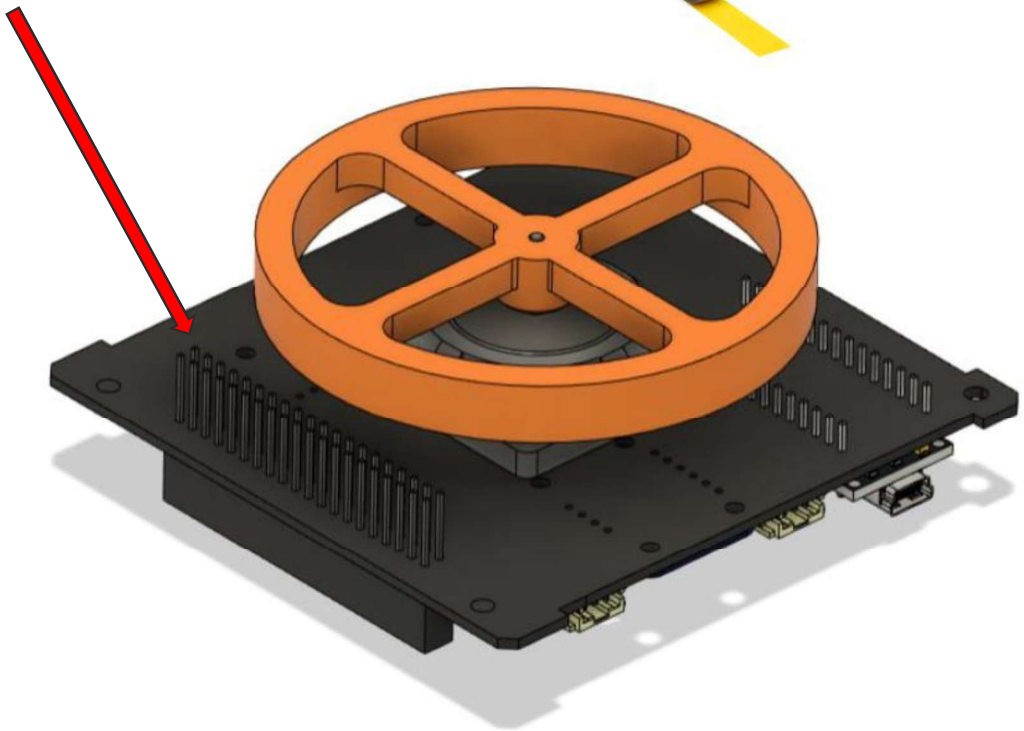
Be careful with the motor wires, unroll them from the motor case and plug them into the connector.



Satellite Set-up - ADCS

Let’s insulate the PC-104 connector pins with kapton tape

Use the same tape to secure motor wires to the board so that they don’t hit the wheel or get close to exposed pins.

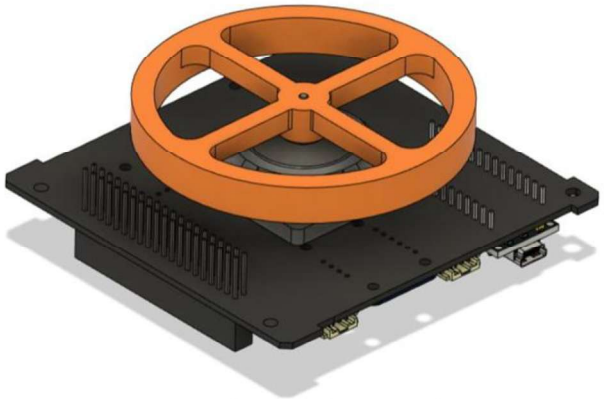


Satellite Set-up - Bottom side

4xCBSTSR M3x10

4x20mm Spacers

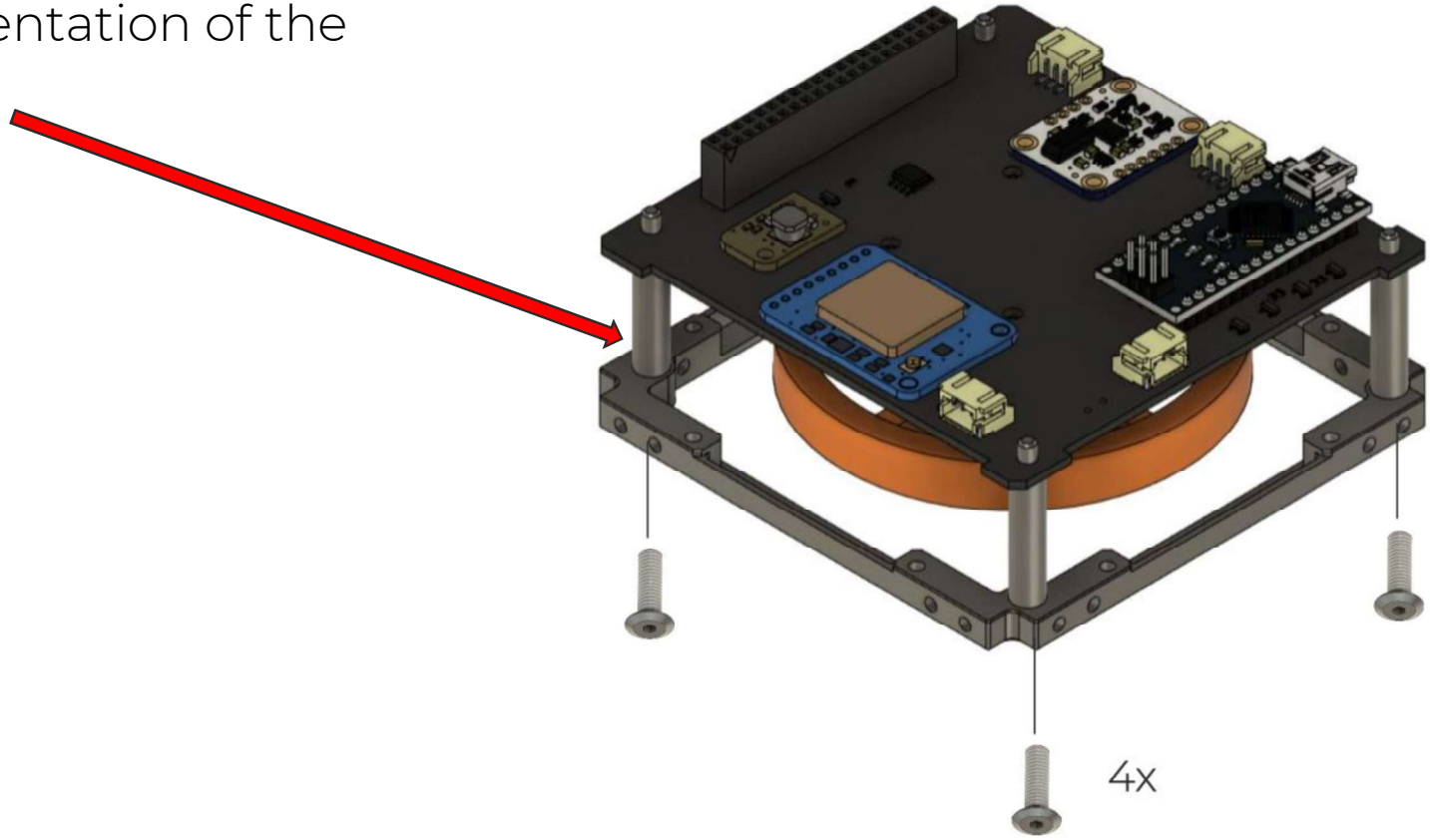
1xVRS006M



1x ADCS Board

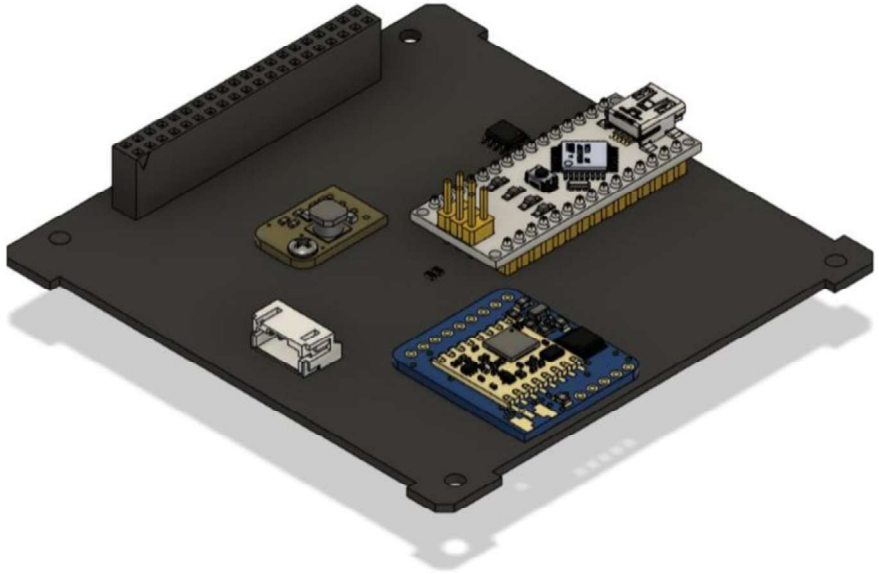
Satellite Set-up - Bottom side

Look at the orientation of the bottom frame!



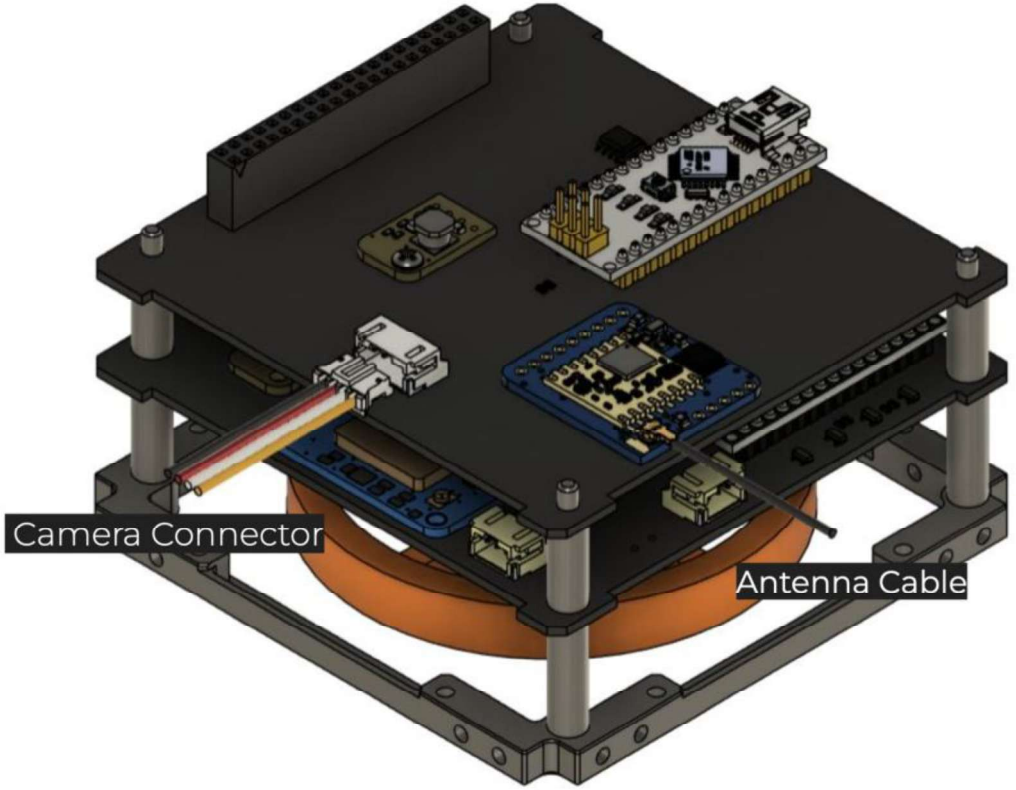
Satellite Set-up - OBC and ADCS Stack

4x15.3mm Spacers



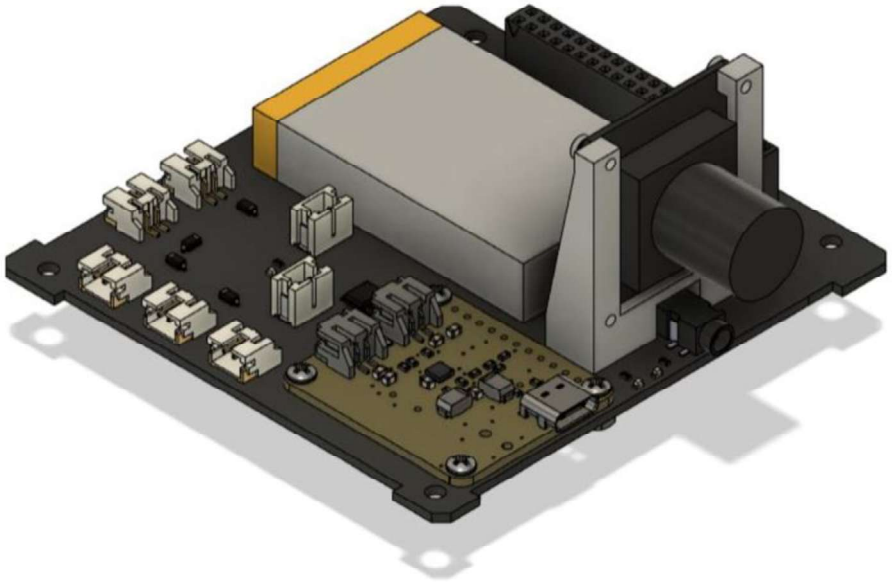
1x OBC Board

Satellite Set-up - OBC and ADCS Stack



Satellite Set-up - Preparing the Full Stack

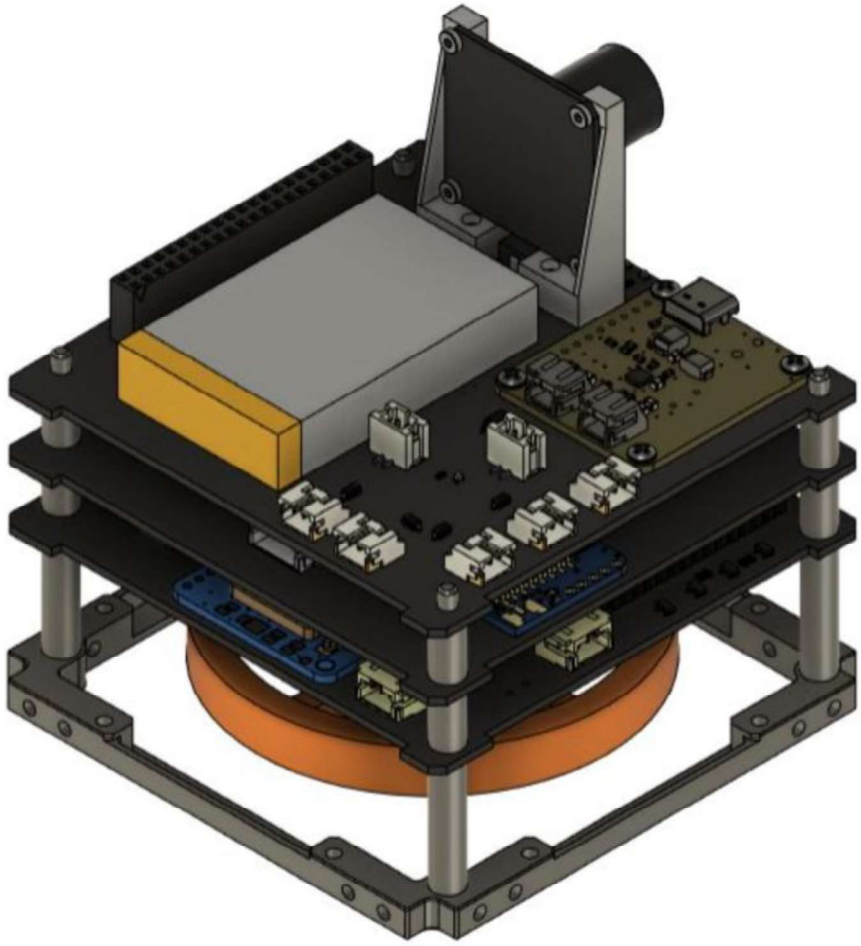
4x15.3mm Spacers



1x EPS Board + Camera

Satellite Set-up - Full Stack

Plug the camera connector to the OBC BEFORE placing the EPS on top of it!

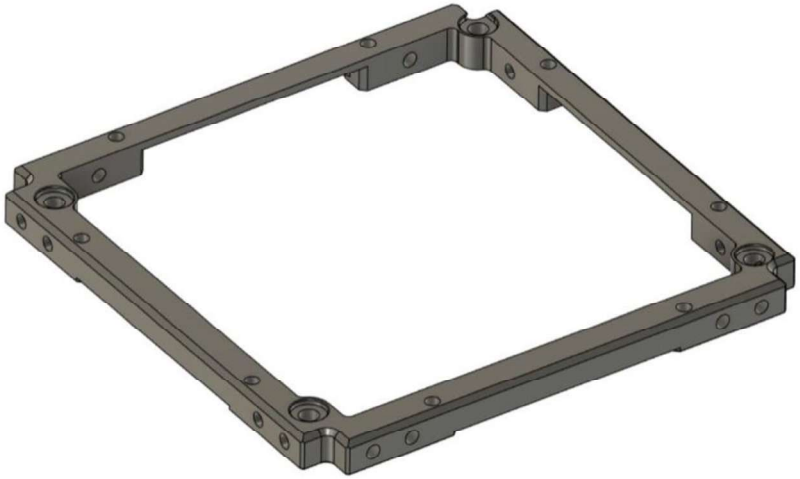


Satellite Set-up - Top Frame

Build the top frame of the 1U in a similar way as for the bottom frame

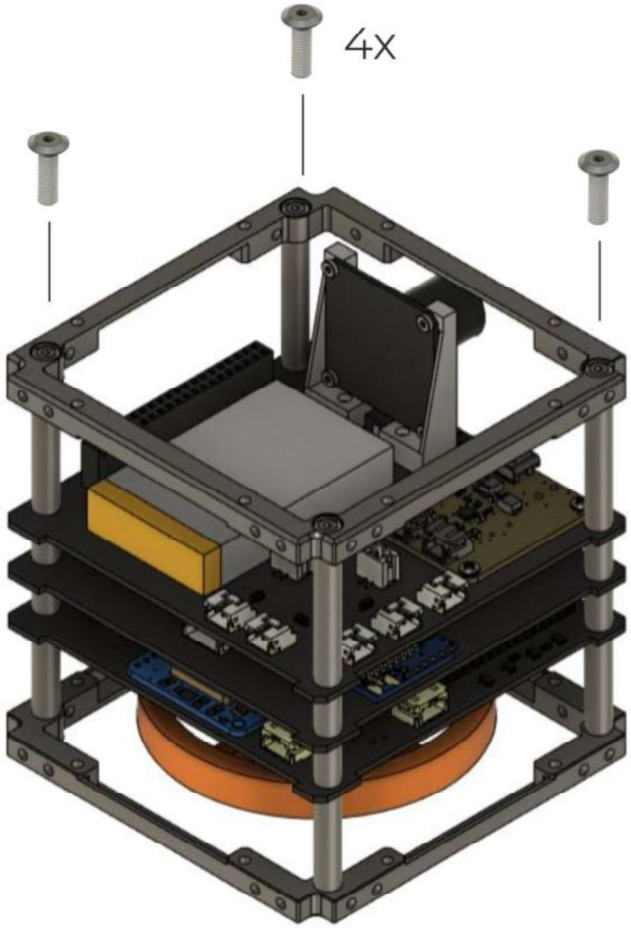
4x30.25 Spacers

4xCBSTSR M3x10



1xVRS006

Satellite Set-up - Top Frame



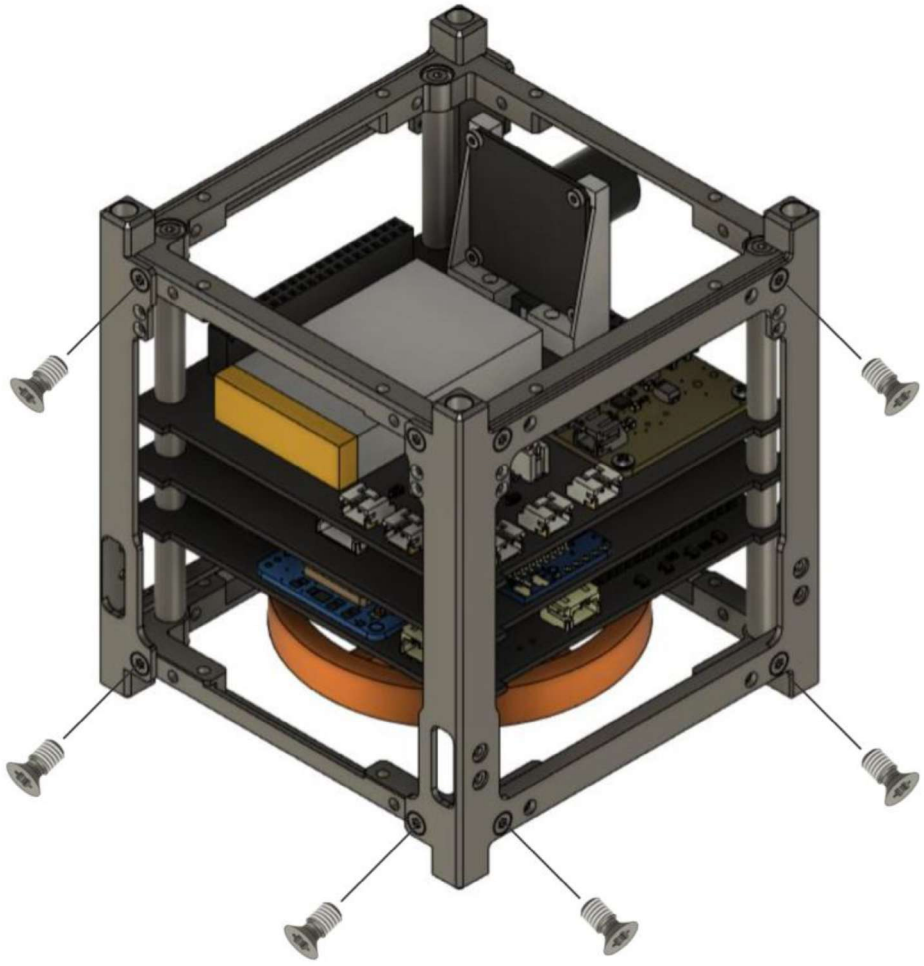
Satellite Set-up - Side Structures

16x TORX M2.5x5

2xVRS001

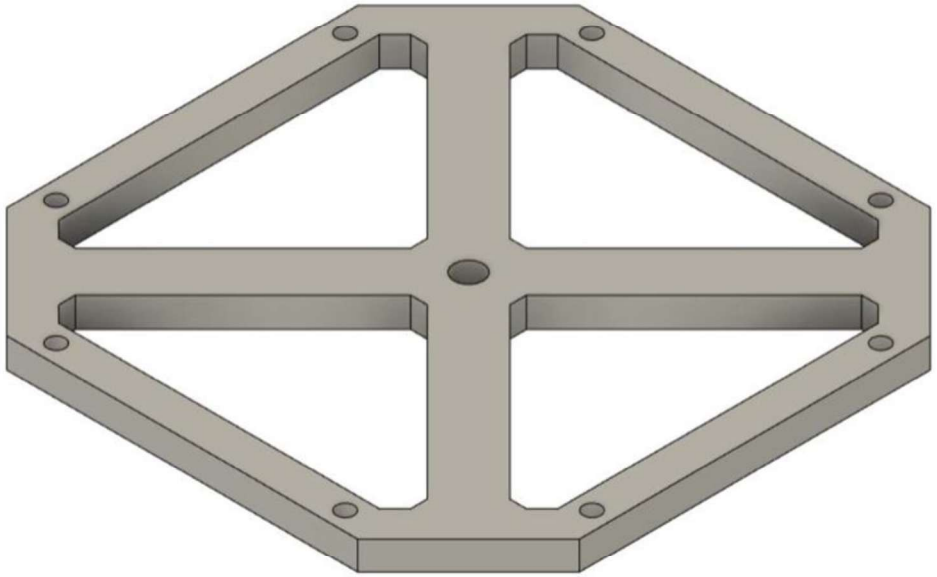
Satellite Set-up - Side Structures

Pay attention to the mounting direction of the side structures



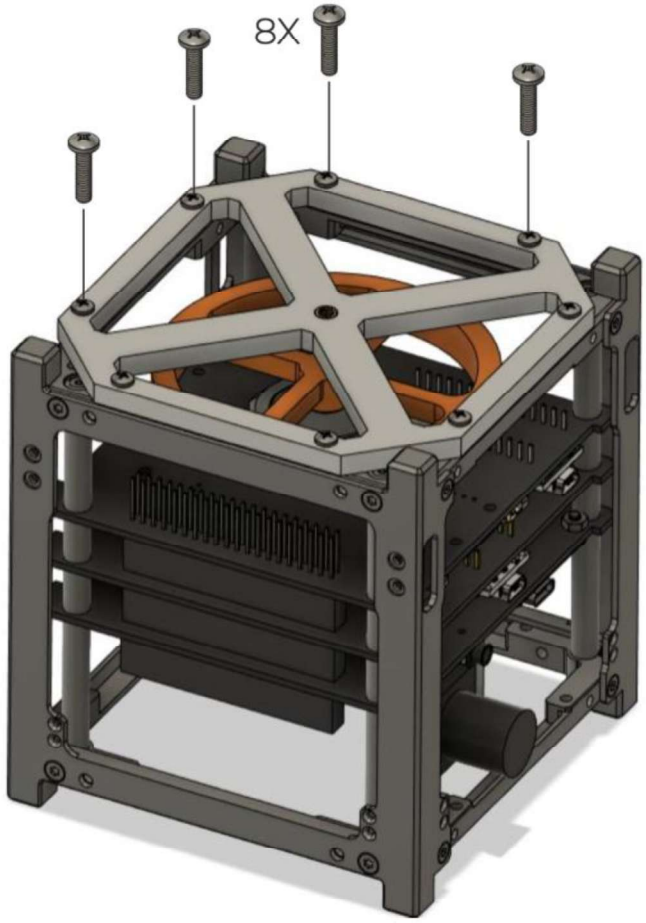
Satellite Set-up - Bottom Guard

8xM2.5x10



1x Support

Satellite Set-up - Bottom Guard



Satellite Set-up - Boards Check

Before continuing with the satellite mechanical set-up, let's make sure our boards can be programmed!

Remember:

- **Arduino Nano** - OBC & Ground Station
- **Arduino Nano Every** - ADCS

Satellite Set-up - Boards Check

Hardware side:

- Connect the USB C to the EPS and plug it in your PC.
- Now remove the RBF.

Software side:

- Open a new sketch in Arduino IDE - do not write any code in it

```
1  void setup() {
2    // put your setup code here, to run once:
3
4  }
5
6  void loop() {
7    // put your main code here, to run repeatedly:
8
9  }
10
```

Satellite Set-up - Boards Check

Software side:

- Open a new sketch in Arduino IDE - do not write any code in it
- Connect and upload the empty code on the OBC first.
- Then, disconnect the OBC cable.
- Connect and upload the empty code on the ADCS.

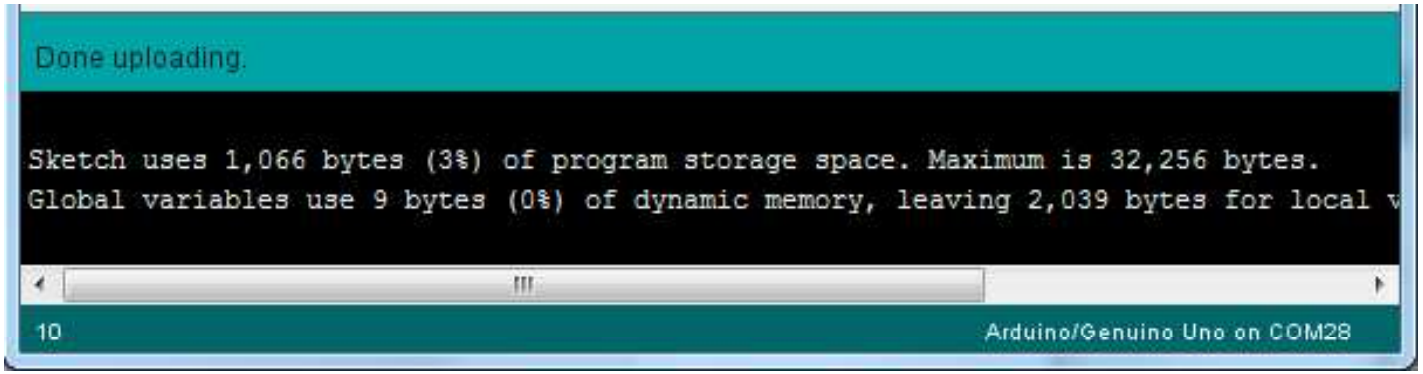
```
1  void setup() {
2    // put your setup code here, to run once:
3
4  }
5
6  void loop() {
7    // put your main code here, to run repeatedly:
8
9  }
10
```

Satellite Set-up - Boards Check

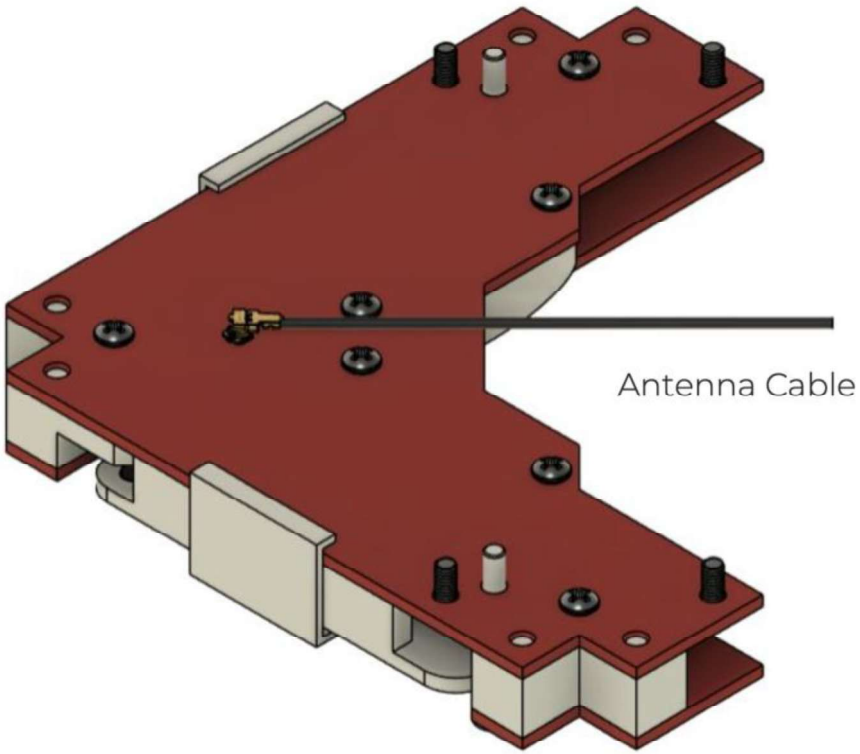
Hardware side:

- Plug the RBF back in.
- Remove the remaining Arduino cable.
- Remove the USB C cable from the EPS.

Now we know that both OBC and the ADCS can be programmed!

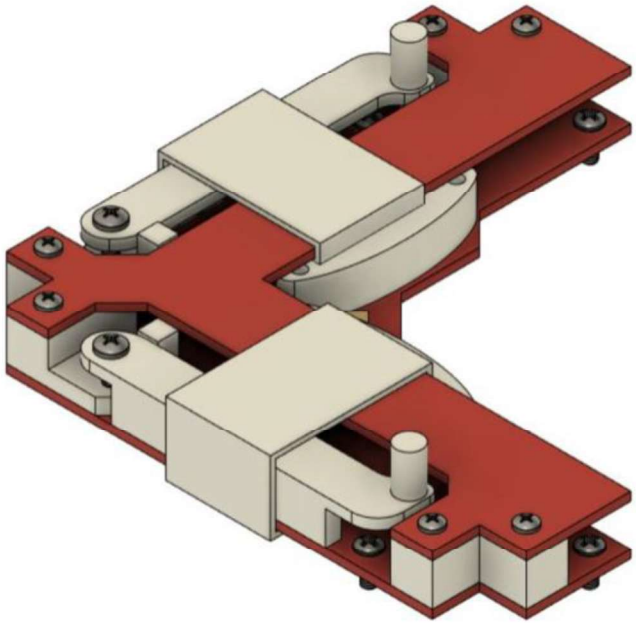


Satellite Set-up - Antenna OBC Connection



Satellite Set-up - Antenna

2xM2.5x6

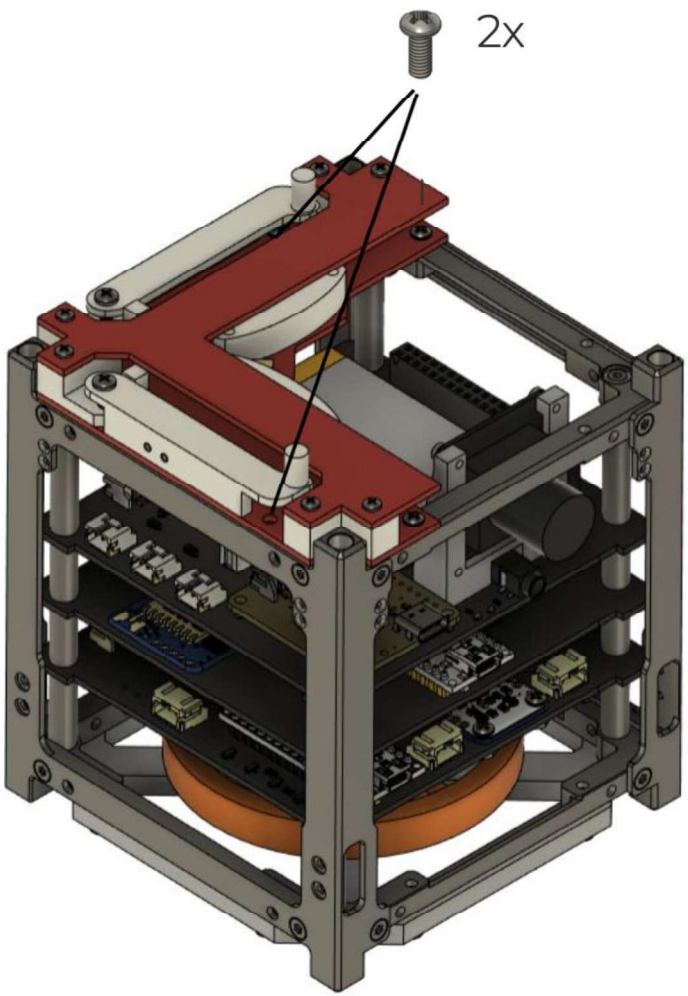


1xDeployable Antenna

Satellite Set-up - Antenna

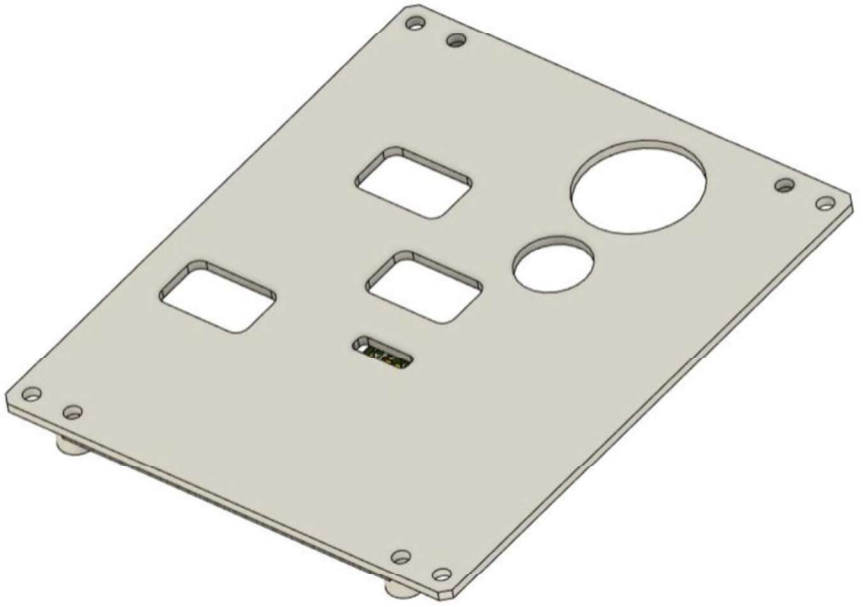
To insert the M2.5x6, open manually deploy the antennas!

Once done, close them back with the pins.



Satellite Set-up - Side Panel

4xM2.5x10

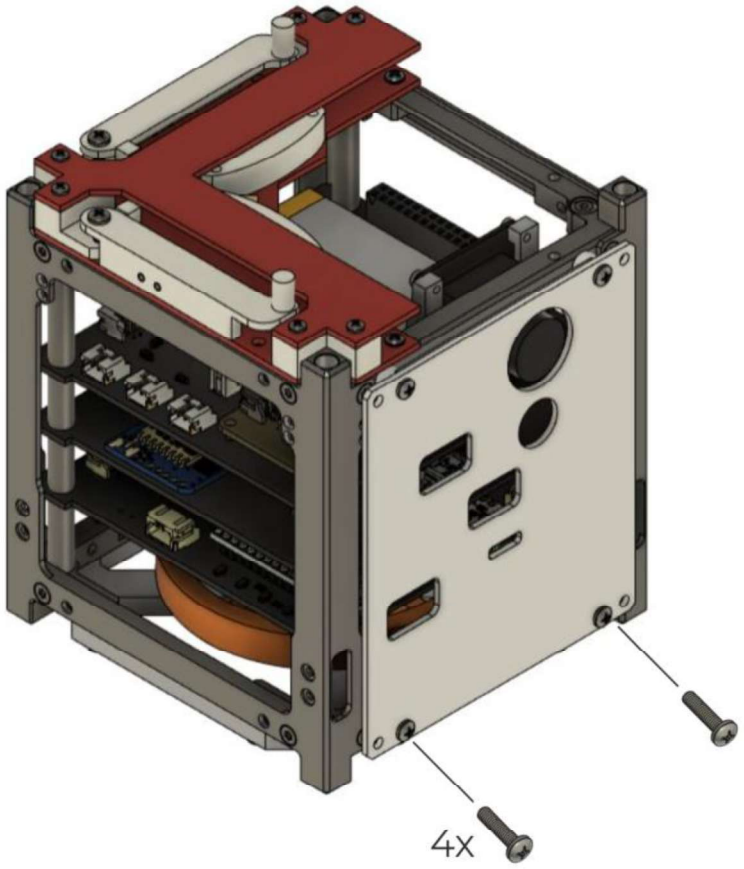


1x Side Panel

Satellite Set-up - Side Panel

To correctly place the panel, we are going to need back spacers!

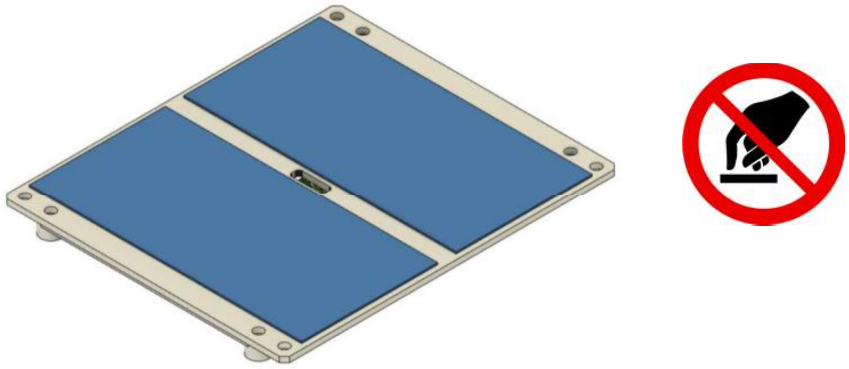
After placing the side panel, take some kapton and label EPS, ADCS, RBF and OBC with a marker.



Satellite Set-up - Solar Panels

Solar panel cells are extremely fragile and tend to be broken easily if touched.

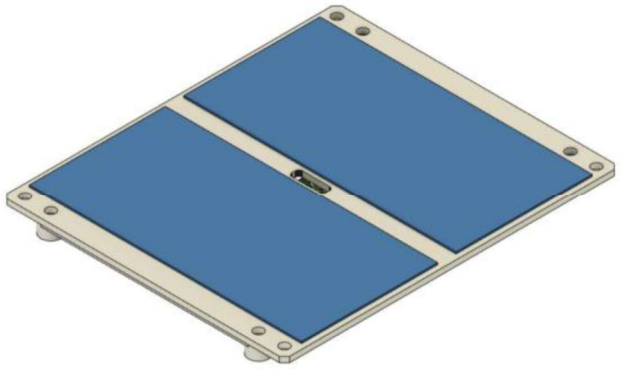
- Avoid touching them directly with your fingers.
- Be careful not to hit them with tools.
- Most importantly, do not place the faces with solar panels directly on the table or onto other components.



Satellite Set-up - Solar Panels

Before mounting the solar panels onto the satellite, it's essential to ensure their functionality.

- Begin by using a multimeter to measure voltage output, checking each panel individually using the torchlight.
- Additionally, inspect the panels to identify any physical defects or anomalies.



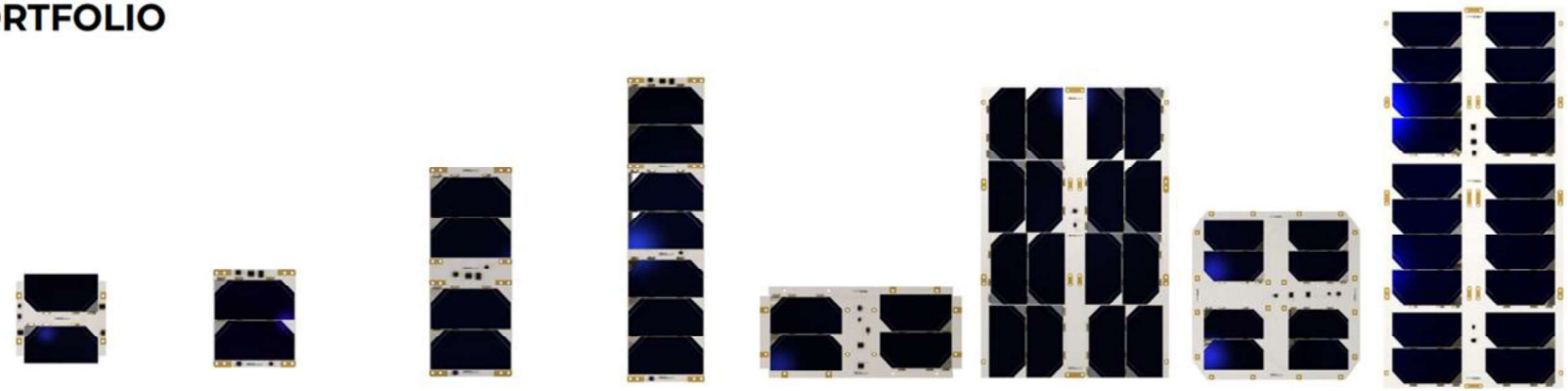
Space Grade Solar Panels

If we consider a double row of solar cells on each face of a CubeSat, the number of cells may be distributed as follows:

- **1U CubeSat**: A 1U CubeSat with a double row of solar cells might have around 6 to 12 solar cells.
- **2U CubeSat**: For a 2U CubeSat, a double row could result in approximately 12 to 24 solar cells.
- **3U CubeSat**: A double row on a 3U CubeSat might feature around 18 to 36 solar cells.
- **6U CubeSat**: On a 6U CubeSat, a double row configuration could include approximately 36 to 60 solar cells.
- **12U CubeSat**: For a 12U CubeSat, with a double row, you might find around 60 to 120 solar cells (doployables).

2NDSpace

PRODUCT **PORTFOLIO**



	CORE-01Z	CORE-01	CORE-02	CORE-03	CORE-06Z	CORE-06	CORE-12Z	CORE-16
Peak Power [W]	2.5	5	5	8,75	5	20	10	25
Suggested for	1U/2U/3U	2U/3U	2U/3U	3U/6U	6U	6U/12U	12U/16U	16U
Form Factor	1x1	1x1	1x2	1x3	2x1	2x3	2x2	2x4
Weight [g]	38	35	75	118	91	289	195	387

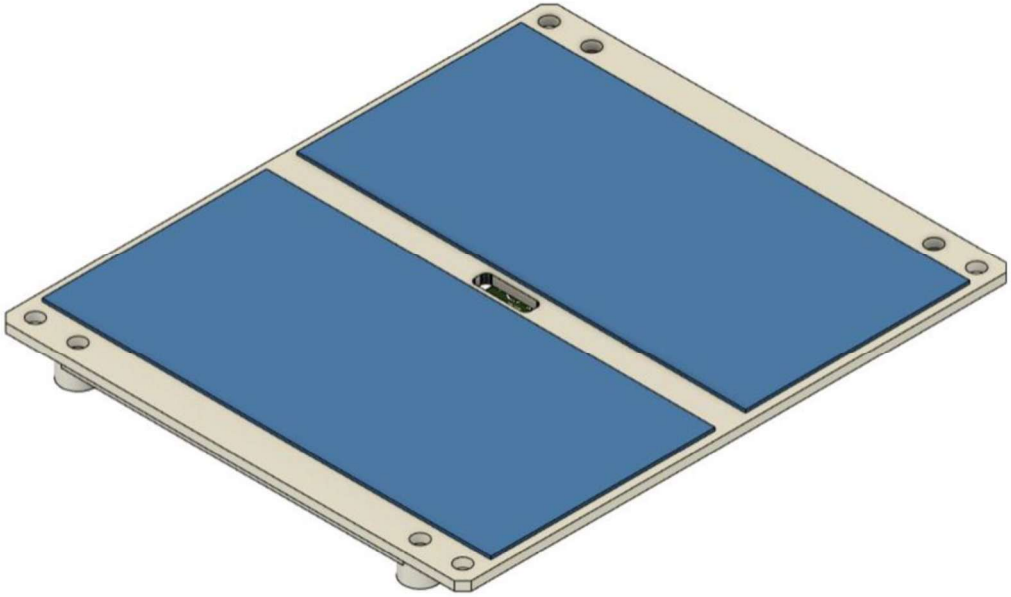
QUALIFICATION AND **ACCEPTANCE TEST**

	Functional/AIT	Electrical Test/Flash	Vibration test	Mechanical Shock	TVAC Test
Qualification Test	✓	✓	NASA GEVS: GSFC-STD-7000A ESA ECSS-E-ST-10-03C	NASA GEVS: GSFC-STD-7000A ESA ECSS-E-ST-10-03C	NASA GEVS: GSFC-STD-7000A ESA ECSS-E-ST-10-03C
Acceptance Test	✓	✓			

2NDSpace

Satellite Set-up - Solar Panels

12xM2.5x10



3x Solar Panels

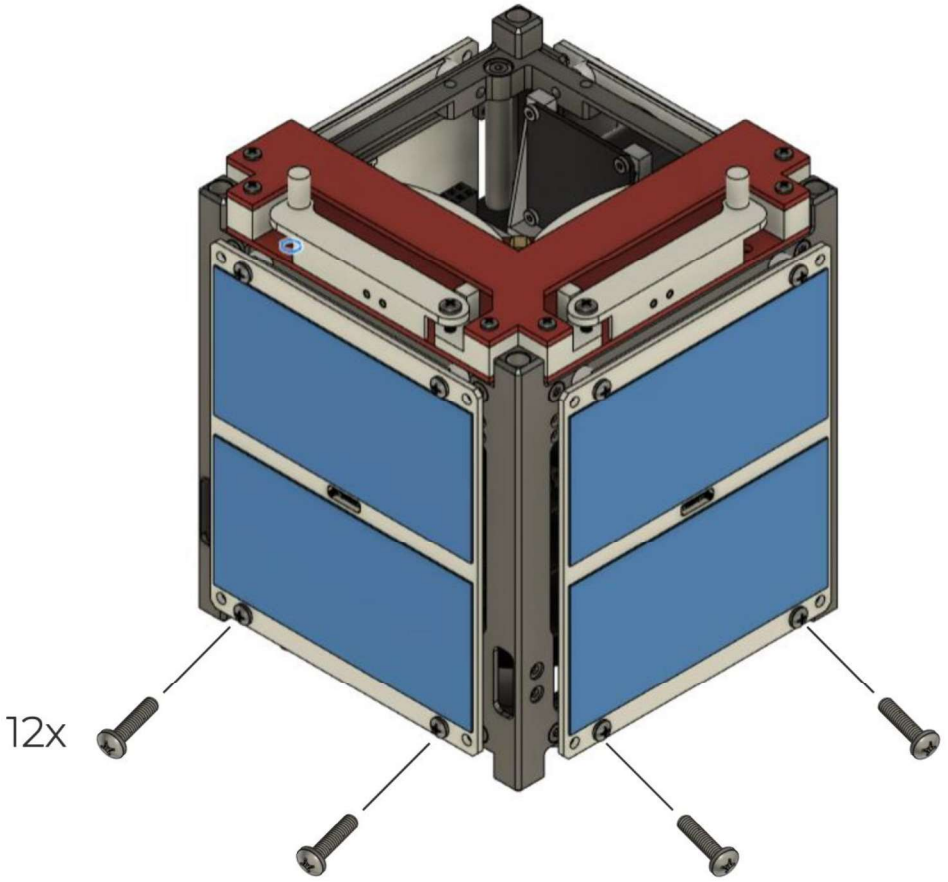
Satellite Set-up - Solar Panels

Before placing the solar panels we need to:

- Connect the solar cells outputs to the EPS.
- Connect the sun sensors to the ADCS.

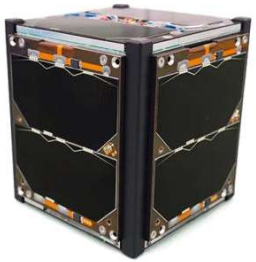
Refer to the EPS and ADCS schematics to understand where to connect each cable.

Use the spacers behind the panels to place them on the satellite.



Satellite Set-up - Full Satellite

Congratulations, you built the entire 1U Cubesat!



Satellite Set-up - Discussion

Satellite Design & Integration

Hands-on 4

2NDSpace

OBC Exercises

Let’s start coding the On Board Computer (OBC):

- Open a *new sketch* and save it as OBC_Exercises.

OBC Exercises

Let's start coding the On Board Computer (OBC):

- Open a *new sketch* and save it as OBC_Exercises.
- Write an OBC program that displays the time elapsed (since the OBC is turned on) in seconds via serial monitor.

The output should look like:

TIME = 1s

TIME = 2s

TIME = 3s

...

OBC Exercises

Let's start coding the On Board Computer (OBC):

- Open a *new sketch* and save it as OBC_Exercises.
- Write an OBC program that displays the time elapsed (since the OBC is turned on) in seconds via serial monitor.

The output should look like:

TIME = 1s

TIME = 2s

TIME = 3s

...

- Once done, include minutes in the display (remember that when we get to 59s we add a minute, resetting the seconds).

The output should look like:

TIME = 0m 25s

...

OBC Exercises

- Request the time from the OBC through input (Serial.read) of the character 'T'. Arduino does not display anything via serial until requested (only one reading). The time counting must occur in the background, regardless of when I ask for 'T'.

OBC Exercises

- Request the time from the OBC through input (Serial.read) of the character 'T'. Arduino does not display anything via serial until requested (only one reading). The time counting must occur in the background, regardless of when I ask for 'T'.
- Find a way to send the character 'R' and reset the time (not Arduino) so that the second time I request 'T', I will have a lower value than before.

OBC Exercises - Analog Sensors

Below you will find the correct way to set up an analog sensor.

As you can see, in contrast to digital sensors, with analog sensors, **there's no need** to use the **pinMode(name, INPUT)** function.

```
1  int sensorPin = A0;
2
3  void setup() {
4    Serial.begin(9600);
5  }
6
7  void loop() {
8    int reading = analogRead(sensorPi
9    Serial.print("Sensor Reading: ");
10   Serial.println(reading);
11   delay(1000);
12 }
```

OBC Exercises

- The OBC has a temperature sensor, **LM35**, which is an analog sensor connected to pin A1.
- Look at the online datasheet of the LM35 to understand how it reads temperature and find a way to display it in the serial monitor every 2 seconds.

The output should look like:

TEMPOBC = 26.2 deg

Product Folder | Order Now | Technical Documents | Tools & Software | Support & Community

TEXAS INSTRUMENTS

LM35

SNIS159H – AUGUST 1999 – REVISED DECEMBER 2017

LM35 Precision Centigrade Temperature Sensors

1 Features **3 Description**

OBC Exercises

- The OBC has a temperature sensor, **LM35**, which is an analog sensor connected to pin A1.
- Look at the online datasheet of the LM35 to understand how it reads temperature and find a way to display it (as a float) in the serial monitor every 2 seconds.

The output should look like:
TEMPOBC = 26.2 deg

- Find a way to request the temperature using the string 'TEMPOBC' and display it on the serial monitor when requested.

OBC Exercises

- The OBC has a temperature sensor, **LM35**, which is an analog sensor connected to pin A1.
- Look at the online datasheet of the LM35 to understand how it reads temperature and find a way to display it (as a float) in the serial monitor every 2 seconds.

The output should look like:
TEMPOBC = 26.2 deg

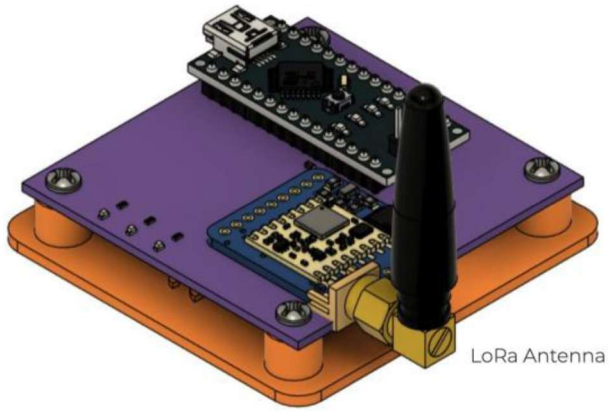
- Find a way to request the temperature using the string 'TEMPOBC' and display it on the serial monitor when requested.
- Add the the possibly of requesting the commands “TIMEOBC”, “RTIMEOBC”,“TEMPOBC” separately in the same Arduino code, dedicating a function for each of them to be called in the void loop().

OBC/GS Exercises - LoRa

Now, let's shift our focus from the CubeSat to the ground station.

We will explore data transmission by uploading code exclusively to the ground station.

My ground station will serve as the receiver for all your ground station messages, as there is no cryptography involved.



OBC/GS Exercises - LoRa

- Find the appropriate LoRa example (File > Examples > LoRa) for transmitting and open it.
- Always use the LoRa.print**ln**() function to send messages to the receiving ground station.
- In the example you will find a counter, remove it so that you send only one message (one every 5 seconds) - stick to this delay time.
- The correct frequency is 434E6.
- Send to my GS a message! Keep it short!

OBC/GS Exercises - LoRa

Now, keep the Arduino connected to your ground station. Take your PC, divide your team, with some members staying inside the room and others going outside to move around the area, checking the range of communication. Use your smartphone to call your teammates and determine exactly when the communication stops. Ensure that the message you send includes your identification.

OBC/GS Exercises - LoRa

- The code from the previous exercise continuously sends the uploaded message on Arduino. Find a way to input a message in the serial monitor (using Serial.read) and ensure it is sent only once.

OBC/GS Exercises - LoRa

- The code from the previous exercise continuously sends the uploaded message on Arduino. Find a way to input a message in the serial monitor (using Serial.read) and ensure it is sent only once.
- After completing the previous step, figure out how to include a timestamp along with the message.

The output should look like:

[0m 30s] your_message

OBC Exercises

Now, let's configure the LoRa sender on the satellite, ensuring that the RBF is plugged in.

OBC Exercises

Now, let's configure the LoRa sender on the satellite, ensuring that the RBF is plugged in. Let’s upload code to the **OBC ONLY**.

- Transmit the temperature from the OBC via radio, along with a brief message to identify yourself.

The output should look like:
[NAME OF GROUP] | TEMPOBC = 20.4

OBC Exercises

Now, let's configure the LoRa sender on the satellite, ensuring that the RBF is plugged in. Let’s upload code to the **OBC ONLY**.

- Transmit the temperature from the OBC via radio, along with a brief message to identify yourself.

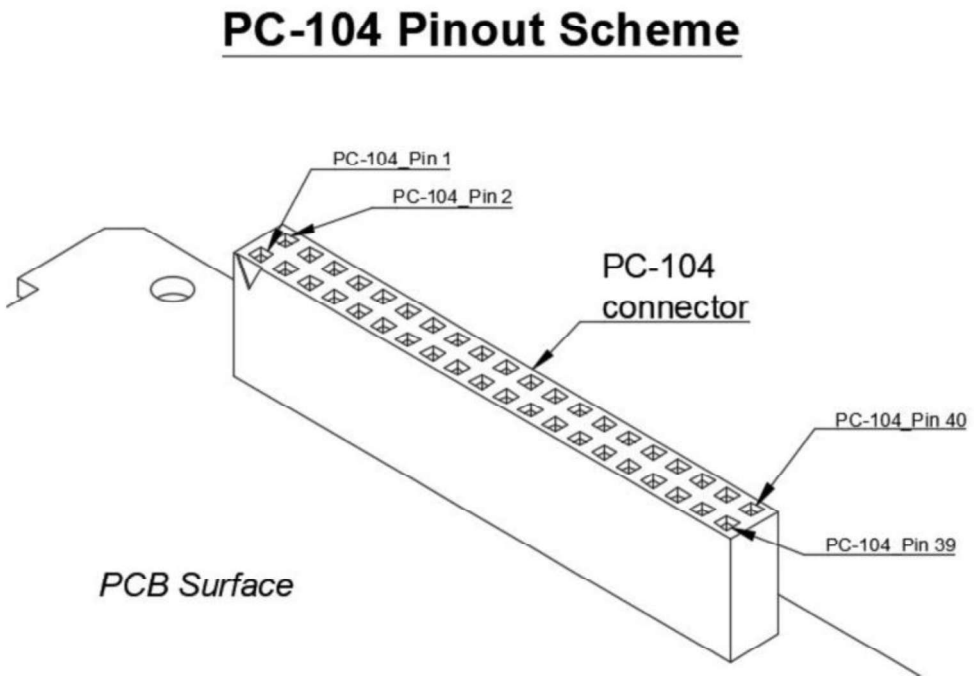
The output should look like:
[NAME OF GROUP] | TEMPOBC = 20.4

- Now, figure out a way to send the temperature when inputting 'TEMPOBC' via serial. Implement the option to send also the time with 'TIME'.

The output should look like:
[NAME OF GROUP] | TEMPOBC = 20.4 | TIME = 0m 34s

OBC Exercises

We can read sensors from other boards around the satellite using the industry-standard PC104 connector.



OBC Exercises

PC-104 Connections

NOTES	Connections EPS	OBC	ADCS	Pin Assignment	PC-104		Pin Assignment
Ground	✓	✓	✓	GND	1	2	NC
OUT of the solar charger	✓	✓	✓	VBUS	3	4	NC
Ground	✓	✓	✓	GND	5	6	NC
				NC	7	8	NC
				NC	9	10	NC
+5V bus from the OBC DC-DC converter	X	✓	X	OBC_5V	11	12	NC
				NC	13	14	NC
Battery Voltage (do not use it for power)	✓	✓	X	VBAT	15	16	NC
				NC	17	18	NC
ADCS reset signal from the OBC	X	✓	✓	ADCS_RESET	19	20	NC
				NC	21	22	NC
Temperature sensor on ADCS board	X	✓	✓	ADCS_TEMP	23	24	NC
Temperature sensor on EPS board	✓	✓	X	EPS_TEMP	25	26	NC
				NC	27	28	NC
ADCS +5V DC-DC ENABLE	X	✓	✓	ADCS_EN	29	30	NC
ADCS - OBC Serial port	X	✓	✓	RX_ADCS_TX_OBC	31	32	NC
ADCS - OBC Serial port	X	✓	✓	TX_ADCS_RX_OBC	33	34	NC
Ground	✓	✓	✓	GND	35	36	NC
OUT of the solar charger	✓	✓	✓	VBUS	37	38	NC
Ground	✓	✓	✓	GND	39	40	NC

OBC Exercises

- Now, referring to the OBC schematics, locate the connection of the EPS temperature sensor on the OBC (no need to send it via LoRa). Write a code to read the temperature of the EPS - the output in your serial monitor should look like: 'TEMPEPS = 20.6.' The EPS utilizes the same temperature sensor as the OBC (LM35).

OBC Exercises

- Now, referring to the OBC schematics, locate the connection of the EPS temperature sensor on the OBC (no need to send it via LoRa). Write a code to read the temperature of the EPS - the output in your serial monitor should look like: 'TEMPEPS = 20.6.' The EPS utilizes the same temperature sensor as the OBC (LM35).
- Figure out how to request 'TEMPOBC' and 'TEMPEPS' via serial so that Arduino provides the desired information when asked. Remember to create separate functions for each.

OBC Exercises

- Now, referring to the OBC schematics, locate the connection of the EPS temperature sensor on the OBC (no need to send it via LoRa). Write a code to read the temperature of the EPS - the output in your serial monitor should look like: 'TEMPEPS = 20.6.' The EPS utilizes the same temperature sensor as the OBC (LM35).
- Figure out how to request 'TEMPOBC,' 'TEMPEPS,' and 'TIME' via serial so that Arduino provides the desired information when asked. Remember to create separate functions for each.
- Create a function, maxTemp(), that compares two temperatures. Invoke the function by entering 'TEMPCOMPARISON' via serial.

The output should should look like:
TEMPx is greater than TEMPy

OBC Exercises

- Include in the function above the absolute difference between the temperatures.

The output should look like:

TEMPx is greater than TEMPy - Difference = 0.2 deg

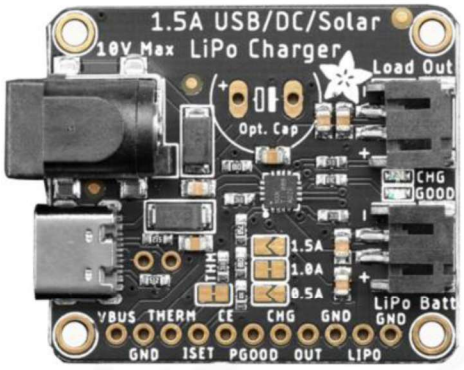
OBC Exercises

- Include in the function above the absolute difference between the temperatures.

The output should look like:

TEMPx is greater than TEMPy - Difference = 0.2 deg

- Refer to the OBC schematics to determine how to read the battery voltage (VBAT) using the **Adafruit 4755 solar charger**. Consult the datasheet online for accurate calculations of the battery voltage. Enable it to be called via serial by typing 'VBAT'.



Satellite Design & Integration

Hands-on 5

2NDSpace

OBC Exercises

- Send to my ground station all your readings via LoRa separately by typing TEMPOBC, TEMPEPS, TIME, TEMPCOMPARISON and VBAT.

The output should look like:

[Team Name] TEMPEPS = 24.20 deg if you typed “TEMPEPS” or

[Team Name] VBAT = 3.67 V if you typed “VBAT” …

OBC Exercises

- Send to my ground station all your readings via LoRa separately by typing TEMPOBC, TEMPEPS, TIME, TEMPCOMPARISON and VBAT.

The output should look like:

[Team Name] TEMPEPS = 24.20 deg if you typed “TEMPEPS” or

[Team Name] VBAT = 3.67 V if you typed “VBAT” …

- Now, write a new function called *sendAll()* that is capable of sending all the readings in one line. This function has to be activated when typing “ALL”.

The output should look like:

[Team Name] TEMPOBC | TEMPEPS = 24.20 deg | VBAT = 3.67 V

OBC Exercises

Let's integrate *warnings* into the satellite!

While a satellite remains unmodifiable once launched, it can still be operated to some extent by a ground station.

In addition to ADCS control and operative modes, we can leverage sensor data to monitor the behavior of subsystems.

If the battery level is too low or the temperature is excessively high, we can deactivate certain systems. These decisions are typically made autonomously upon receiving a warning.

OBC Exercises

Warnings are functions that need to run in the background, constantly monitoring specific conditions. Even if the satellite's status is not actively queried (from the serial monitor), these warnings should appear when such conditions undergo a change.

- Create a warning function that checks the temperatures of both the OBC and the EPS. If either of them detects a temperature higher than 30 degrees Celsius, display the following message in your serial monitor:

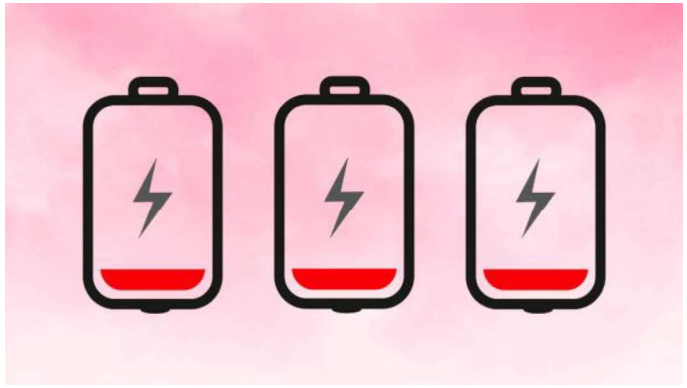
[3m 43s] WARNING! TEMPOBC is too high: 35 deg! or
[3m 43s] WARNING! TEMPEPS is too high: 35 deg!

We can use the rework to test it out!

OBC Exercises

- Create a warning function that checks the voltage of the battery on the EPS. If the value is found to be lower than 3.6 V, display the following message in your serial monitor:

[3m 43s] WARNING! VBAT is too low: 3.5 V!



OBC Exercises

Add to the code the possibility of typing “STATUS” to get an output that looks like this:

- [3m 43s] NOMINAL — if no warnings are found
- [3m 43s] WARNING! TEMPxx is too high: 35 deg! — if temperature is too high
- [3m 43s] WARNING! VBAT is too low: 3.5 V! — if battery is too low

OBC Exercises

Add to the code the possibility of typing “STATUS” to get an output that looks like this:

- [3m 43s] NOMINAL — if no warnings are found
- [3m 43s] WARNING! TEMPxx is too high: 35 deg! — if temperature is too high
- [3m 43s] WARNING! VBAT is too low: 3.5 V! — if battery is too low

Now, in the *sendAll()* function include the STATUS of the satellite in two lines.

[Team Name] TEMPOBC | TEMPEPS = 24.20 deg | VBAT = 3.67 V |
[Team Name] STATUS = one_of_the_cases_above

OBC Exercises

- Implement a code to monitor the STATUS so that, even when we don't explicitly request 'ALL' or 'STATUS,' if the satellite detects a warning, it will send a message to the ground station via LoRa.

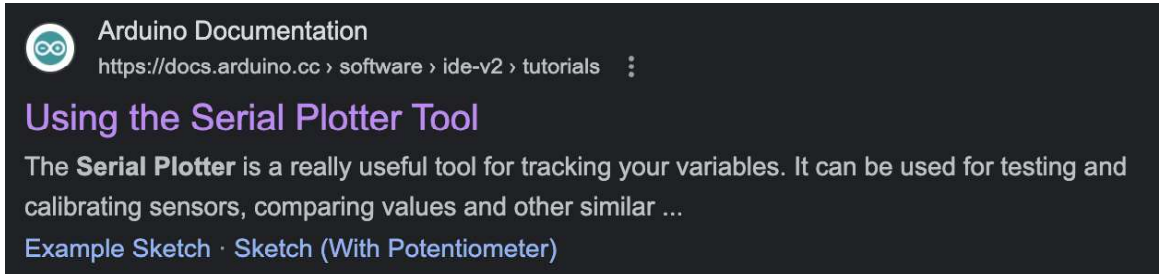
[Team Name] STATUS = WARNING! VBAT is too low: 3.5 V!

When you are done **SAVE EX_5_SENDALL_WARNINGS_INCLUDED** from GitHub!

Back to the OBC!

To ensure consistency in sensor readings, let's explore the Serial Plotter in the Arduino IDE. **There is no need for LoRa. Let’s test this on a new sketch.**

Let’s have a look at the Serial Plotter on the Arduino IDE. Refer on its documentation which is accessible online on the Arduino website.



Back to the OBC!

To ensure consistency in sensor readings, let's explore the Serial Plotter in the Arduino IDE. **There is no need for LoRa. Let’s test this on a new sketch.**

Let’s have a look at the Serial Plotter on the Arduino IDE. Refer on its documentation which is accessible online on the Arduino website.

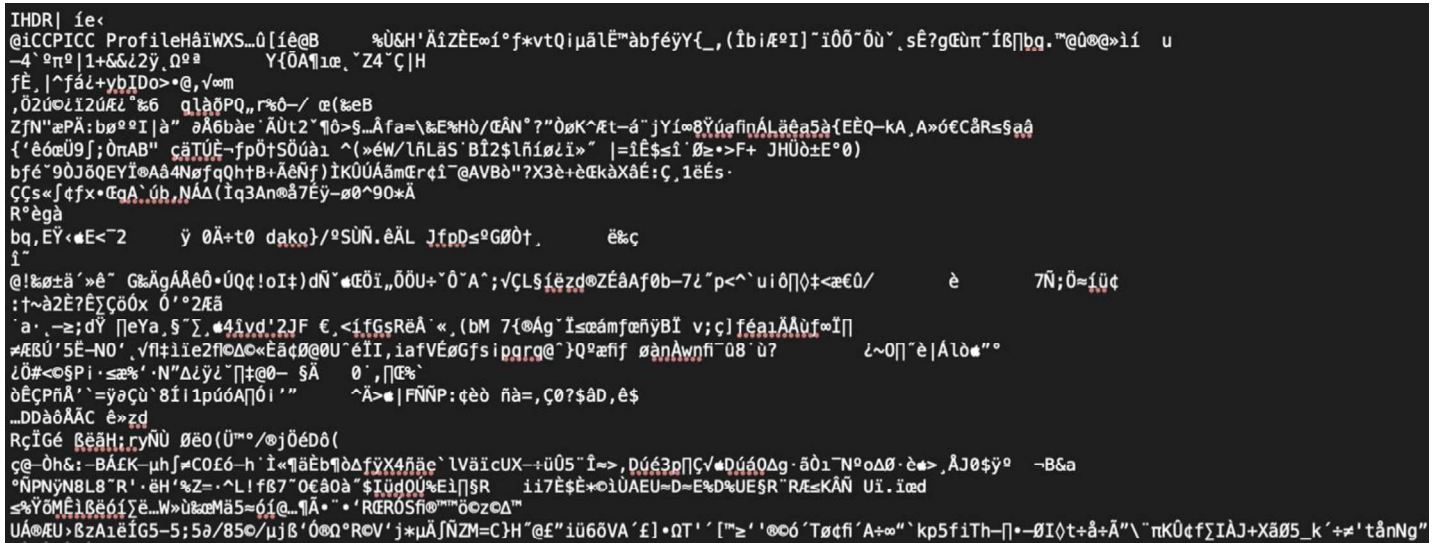
- Utilize the serial plotter to graphically represent all readings, including TEMPOBC, TEMPEPS, and VBAT.
- Remember that the reading must be continuous to correctly plot the values.
- Find a way to calculate the variability of the sensor readings within a 5 seconds as sample time.

OBC Exercises - Camera Challenge

The satellite has a serial camera module called **Adafruit VC0706**.

By referring to its online documentation:

- Create a **new sketch**.
- Refer to the OBC schematics to understand the camera connection to Arduino.
- Create a code that allows you to take a picture and show it in its “text” form. We will see how to convert it to JPG!



SPACE FACULTY

2NDSpace

Satellite Design & Integration

Hands-on 6

2NDSpace

Camera & CoolTerm

To capture a picture with the OBC camera we need to open CoolTerm.

Let’s have a look at the tutorial on GitHub - Code Solutions Day6!

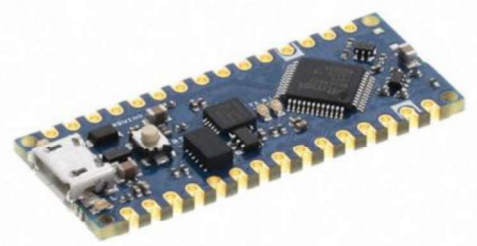
ADCS Exercise

Now, let's focus on the ADCS!

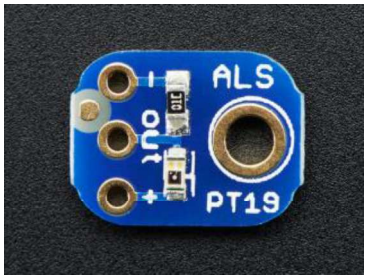
Remember that the ADCS has an Arduino Nano Every (USB Micro).

IMPORTANT:

- **Keep the USB C connected to charge the battery for all the exercises since the ADCS draws a lot of current.**
- **DO NOT UPLOAD ANY OF THE OBC PROGRAMS ON THE ADCS.**



ADCS Exercise



CREATE A NEW SKETCH CALLED *ADCS_EXERCISE*

The ADCS can read values from 3 analog sun sensors (**Adafruit ALS-PT19**) placed inside the solar panels:

- Look at the online documentation of the sun sensors and create a function to read one of them first to see what the output looks like in the serial monitor. Refer to the ADCS schematics to see the pin connection of the sensor.

ADCS Exercise

CREATE A NEW SKETCH CALLED *ADCS_EXERCISE*

The ADCS can read values from 3 analog sun sensors (**Adafruit ALS-PT19**) placed inside the solar panels:

- Look at the online documentation of the sun sensors and create a function to read one of them first to see what the output looks like in the serial monitor. Refer to the ADCS schematics to see the pin connection of the sensor.
- Then let's assume the camera has to point to the earth, so the back panel has to face the sun. Create a function that compares the three sun sensors so that the serial monitor output looks like this:

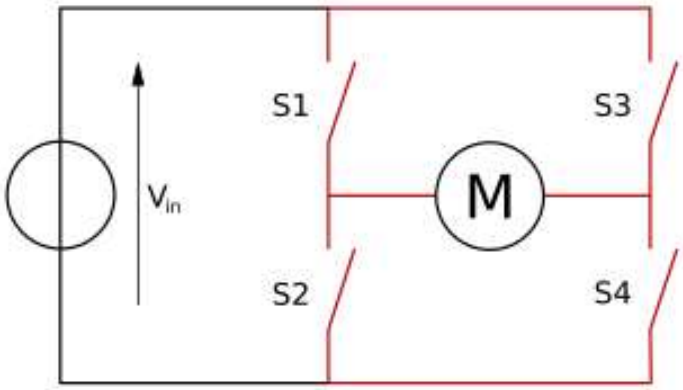
THE CAMERA IS OBSERVING EARTH or
THE SATELLITE POSITION NEEDS TO BE ADJUSTED

- Use the flashlight to check it.

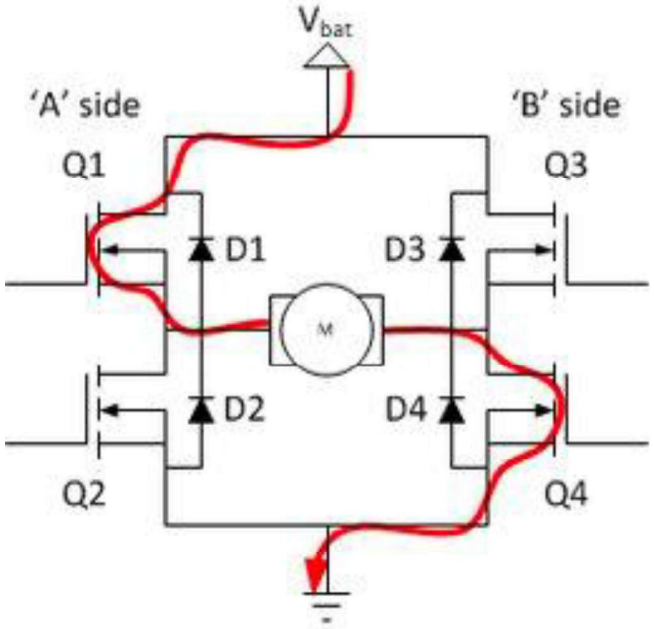
ADCS Exercise

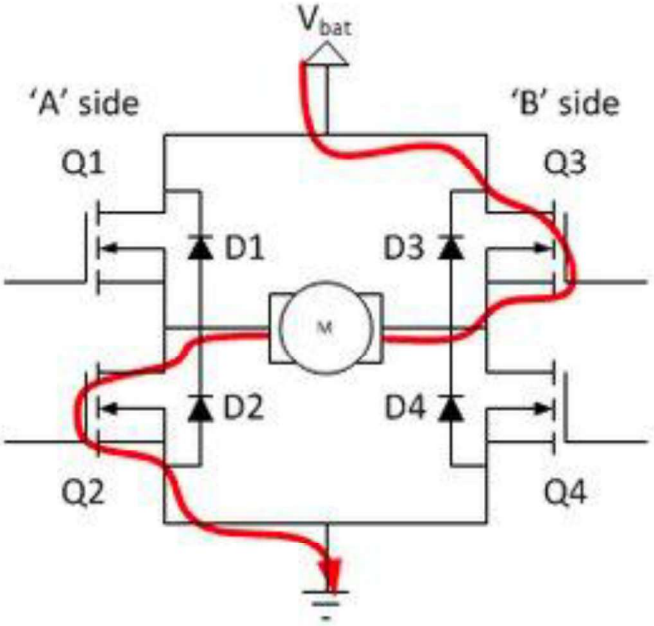
We have left, back, and right sun sensors with respect to the camera. We want to control the motor based on their values. The motor is controlled by a so-called H bridge.

An H-bridge is an electronic circuit crucial for controlling the direction of DC motors. It consists of four switches arranged in an "H" shape, allowing the reversal of voltage polarity across the motor. By selectively closing switch pairs, the H-bridge can control the motor's rotation direction—forward, reverse, or brake.



ADCS Exercise





ADCS Exercise

This is how you correctly drive the motor:

- Set up the **pinMode as an OUTPUT**.
- Always use ***analogWrite()***.
- **Write a speed no higher than 255**.
- Keep a ***delay(100)*** at least.
- **NEVER USE *digitalWrite()***.

```
int motorPin = 9;  // Choose a PWM-enabled pin
int speed = 0;     // Speed variable (0 to 255)

void setup() {
  pinMode(motorPin, OUTPUT);
}

void loop() {
  // Increase speed
  for (speed = 0; speed <= 255; speed++) {
    analogWrite(motorPin, speed);
    delay(10);
  }

  // Decrease speed
  for (speed = 255; speed >= 0; speed--) {
    analogWrite(motorPin, speed);
    delay(10);
  }
}
```

ADCS Exercise

Write a function that:

- If the left sensor has more light, the motor starts rotating counterclockwise for 1 second.
- If the back sensor has more light, the motor is stopped.
- If the right sensor has more light, the motor starts rotating clockwise for 1 second.
- Include a Serial.print of the motor status (rotating ccw, rotating cw or stopped).

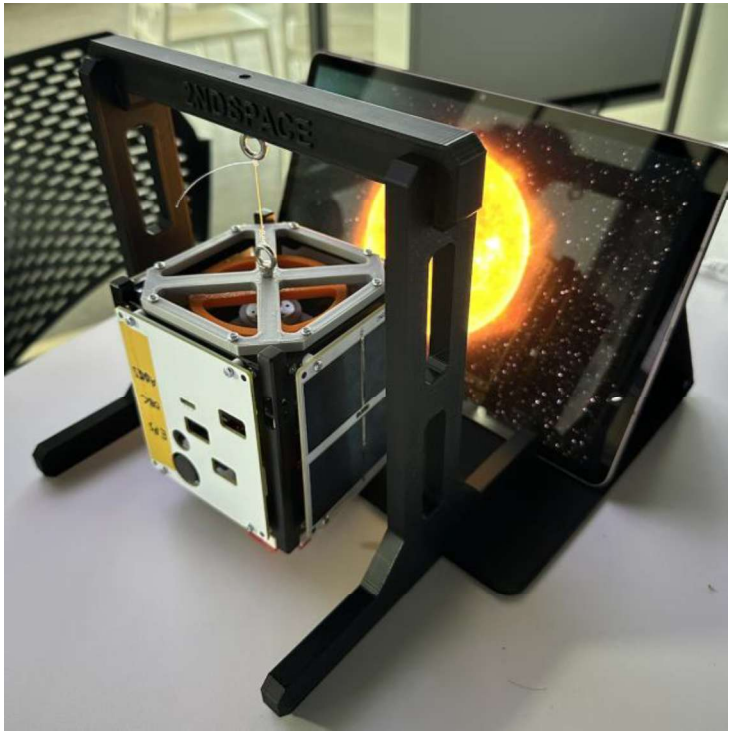
REMEMBER TO USE analogWrite(), NOT digitalWrite()

The maximum speed written has to be 255.

Use a 5 seconds sample time so that the ADCS Arduino doesn't constantly check the sun sensor values.

ADCS Exercise

Test this function by hanging the satellite to the stand using the **eyebolts** and the **fishing line**!
Once you tested it, un-hang the satellite for the next exercise.



ADCS Exercise

Now, comment out the motor and sun sensor parts.

The ADCS has an IMU module called **Adafruit BNO055**, refer to the ADCS schematics for the connections to the Arduino.

- Using online documentation, write a function to read ALL the IMU outputs in the serial monitor.

ADCS Exercise

Now, comment out the motor and sun sensor parts.

The ADCS has an IMU module called Adafruit BNO055, refer to the ADCS schematics for the connections to the Arduino.

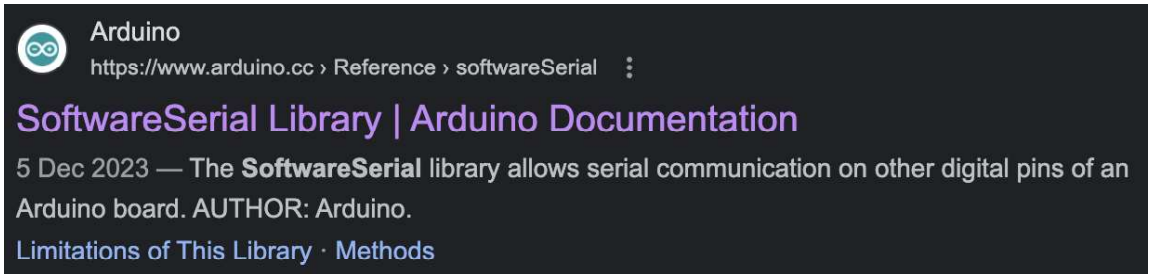
- Using online documentation, write a function to read ALL the IMU outputs in the serial monitor.
- Look at the outputs to find the right threshold for the following exercise.
- Create a function called shake() so that if the satellite is shaken, the motor starts rotating clockwise for 500ms and then counterclockwise for 500ms. The program should not repeat itself unless the satellite is shaken again.

ADCS and OBC Exercise

Although the ADCS Arduino controls its own sensors and motors, the only way to send commands to it or retrieve readings is by connecting it to the OBC Arduino. To achieve this, a dedicated serial communication link between the ADCS and OBC must be established.

- Refer to the ADCS and OBC schematics to identify which pins are used for this serial connection.
- By looking at the online documentation on Software Serial, write two separate codes (**one to be uploaded on the ADCS, and one to be uploaded on the OBC**) to read the temperature of the ADCS (LM35) from the OBC's serial monitor.

DO NOT UPLOAD THE OBC CODE INTO THE ADCS



ADCS and OBC Exercise

Although the ADCS Arduino controls its own sensors and motors, the only way to send commands to it or retrieve readings is by connecting it to the OBC Arduino. To achieve this, a dedicated serial communication link between the ADCS and OBC must be established.

- Refer to the ADCS and OBC schematics to identify which pins are used for this serial connection.
- By looking at the online documentation on **Software Serial**, write two separate codes (**one to be uploaded on the ADCS, and one to be uploaded on the OBC**) to read the temperature of the ADCS (LM35) from the OBC's serial monitor.
- Make this function callable from the OBC by typing 'TEMPADCS' into the serial monitor. **DO NOT UPLOAD THE OBC CODE INTO THE ADCS**

ADCS and OBC Exercise

Write a code that, if we type “SUN” **from OBC’s serial monitor**, we activate the motor movements we created earlier as specified below:

- If the left sensor has more light, the motor starts rotating counterclockwise for 1 second.
- If the back sensor has more light, the motor is stopped.
- If the right sensor has more light, the motor starts rotating clockwise for 1 second.
- Include a Serial.print of the motor status (rotating ccw, rotating cw or stopped).

REMEMBER TO USE analogWrite(), NOT digitalWrite()

The maximum speed written has to be 255.

Use a 5 seconds sample time so that the ADCS Arduino doesn’t constantly check the sun sensor values.

ADCS and OBC Exercise

Include a function called ***detumbling()*** called by typing “DETUMB” that acts like this:

- The motor should move back and forth for 2 seconds in each direction for a total time of 10 seconds and then stops completely. The maximum speed should be 255.

ADCS and OBC Exercise

Include a function called ***detumbling()*** called by typing “DETUMB” that acts like this:

- The motor should move back and forth for 2 seconds in each direction for a total time of 10 seconds and then stops completely. The maximum speed should be 255.

Include also a function that

- If the satellite is shaken, a message should appear on the OBC saying: ANOMALY IN ACCELERATION!

ADCS and OBC Exercise

~~Include a function called ***detumbling()*** called by typing "DETUMB" that acts like this:~~

- ~~The motor should move back and forth for 2 seconds in each direction for a total time of 10 seconds and then stops completely. The maximum speed should be 255.~~

~~Include also a function that~~

- ~~If the satellite is shaken, a message should appear on the OBC saying: ANOMALY IN ACCELERATION!~~

Now, send me all your reading through LoRa:

[2m 23s] [Team Name] TEMPOBC | TEMPEPS | TEMPADCS | VBAT | Camera_pointing_status

Camera_pointing_status = THE CAMERA IS OBSERVING EARTH or THE SATELLITE POSITION NEEDS TO BE ADJUSTED

Satellite Design & Integration

Hands-on 7

2NDSpace

Final Exercise - Challenge!

Configure your entire satellite so that it takes the following commands from your ground stations!

Command	Expected Serial Response	Action Taken
ENADCS	ADCS Enabled	Enables the the ADCS to provide full power - without activating this function the ADCS should not work.
TEMPOBC	TEMPOBC: 23.15°C	Reads temperature on the OBC and reports it via LoRa.
TEMPEPS	TEMPEPS: 23.15°C	Reads temperature on the EPS and reports it via LoRa.
TEMPADCS	TEMPADCS: 23.15°C	Reads temperature on the ADCS and reports it via LoRa.
VBAT	VBAT: 3.72V	Reads battery voltage on the satellite and reports it via LoRa.
CAM		Takes a picture via camera payload and sends it via LoRa.
TIME	[10m 42s]	Sends the time since the boot of the satelltie via LoRa.
IMU	X:359.4 Y:0.69 Z: 7.63	Reads the current orientation of the satellite using the IMU (0-360 deg) and sends it via LoRa.
SUN	S1: 0 S2: 400 S3: 350	Reads the sun sensors values and send them via LoRa.
ADJ	Satellite Rotated	Compares the values of the sun sensors, if the one opposite to the payload camera is the one facing more light, then no action is done. If that's not the case, the satellite should rotate until it reaches this condition.
ALL	***All of the above apart from CAM and ADJ***	Perform all the actions above apart from CAM and ADJ. The output should look like: [2m 23s] TEMPOBC | TEMPEPS | TEMPADCS | VBAT | X:359.4 Y:0.69 Z: 7.63

Final Exercise Solution

- Upload the OBC, ADCS and GS codes into their respective boards from GitHub.
- Change the password in OBC, ADCS and GS code, matching them, so that only your ground station is configured with your satellite.
- Check the code working!

```
String secretKey = "<PASSWORD>";
String inputmsg;
```

Videos

- https://www.youtube.com/watch?v=9vymPdQgoOE
- https://www.youtube.com/watch?v=M_50TM3OeEw
- https://www.youtube.com/watch?v=Z6esOcWrrEE&t=13s
- https://www.youtube.com/watch?v=P9C25Un7xaM